

A Hash Table Without Hash Functions, and How to Get the Most Out of Your Random Bits

William Kuszmaul*

Abstract

This paper considers the basic question of how strong of a probabilistic guarantee can a hash table, storing $n(1 + \Theta(1)) \log n$ -bit key/value pairs, offer? Past work on this question has been bottlenecked by limitations of the known families of hash functions: The only hash tables to achieve failure probabilities less than $1/2^{\text{polylog } n}$ require access to fully-random hash functions—if the same hash tables are implemented using the known explicit families of hash functions, their failure probabilities become $1/\text{poly}(n)$.

本文研究一个基础问题：对于一个存储 n 个 $(1 + \Theta(1)) \log n$ 比特 key/value 对的 hash table，它究竟能提供多强的概率保证？以往关于这一问题的研究，一直受限于已知 hash-function family 的能力边界：目前只有在允许 fully-random hash functions 的情况下，hash table 才能把 failure probability 压到 $1/2^{\text{polylog } n}$ 以下；如果把同样的 hash table 改为使用已知的显式 hash-function families 来实现，其 failure probability 就会退化为 $1/\text{poly}(n)$ 。

To get around these obstacles, we show how to construct a randomized data structure that has the same guarantees as a hash table, but that *avoids the direct use of hash functions*. Building on this, we are able to construct a hash table using $O(n)$ random bits that achieves failure probability $1/n^{1-\epsilon}$ for an arbitrary positive constant ϵ .

为了绕开这一障碍，本文展示了如何构造一种 randomized data structure，它在保证层面与 hash table 等价，但避免了直接使用 *hash functions*。在此基础上，我们构造出了一个只使用 $O(n)$ 个 random bits 的 hash table，其 failure probability 可达 $1/n^{1-\epsilon}$ ，其中 ϵ 是任意给定的正常数。

In fact, we show that this guarantee can even be achieved by a *succinct dictionary*, that is, by a dictionary that uses space within a $1 + o(1)$ factor of the information-theoretic optimum.

更进一步，我们证明这种保证甚至可以在 *succinct dictionary* 中实现，也就是说，可以在仅比 information-theoretic optimum 多出一个 $1 + o(1)$ 因子的空间内完成。

Finally we also construct a succinct hash table whose probabilistic guarantees fall on a different extreme, offering a failure probability of $1/\text{poly}(n)$ while using only $\tilde{O}(\log n)$ random bits. This latter result matches (up to low-order terms) a guarantee previously achieved by Dietzfelbinger et al., but with increased space efficiency and with several surprising technical components.

最后，我们还构造了另一种 succinct hash table，它处在概率保证的另一端：只使用 $\tilde{O}(\log n)$ 个 random bits，同时提供 $1/\text{poly}(n)$ 的 failure probability。这个结果在保证级别上与 Dietzfelbinger 等人的既有结果一致（低阶项意义下），但空间效率更高，而且包含若干出人意料的技术成分。

*MIT CSAIL, kuszmaul@mit.edu. Funded by a Fannie and John Hertz Fellowship and an NSF GRFP Fellowship. This research was also partially sponsored by the United States Air Force Research Laboratory and the United States Air Force Artificial Intelligence Accelerator and was accomplished under Cooperative Agreement Number FA8750-19-2-1000. The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the United States Air Force or the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes notwithstanding any copyright notation herein.

1 Introduction

A *dictionary* is any data structure that supports insertions, deletions, and queries on a set S of up to n *keys*; dictionaries often also allow for a user to store a value associated with each key, which can then be retrieved during queries. Unless stated otherwise, we will assume a machine word of $\Theta(\log n)$ bits, which means that keys/values are also $O(\log n)$ bits. We will also require implicitly that a dictionary should take at most linear space (i.e., $O(n \log n)$ bits) and that a dictionary should be explicit (i.e., it can be initialized in time $O(n)$). In fact, the dictionaries in this paper have the stronger property that they can be initialized in constant time.

dictionary 是一种支持 insert、delete 和 query 的数据结构，维护一个大小至多为 n 的 *keys* 集合 S ；它通常还允许用户为每个 key 存储一个 value，并在查询时返回该 value。除非另行说明，本文假设 machine word 长度为 $\Theta(\log n)$ 比特，因此 key/value 的长度也都是 $O(\log n)$ 。我们还隐含要求 dictionary 只使用线性空间，即 $O(n \log n)$ 比特，并且是 explicit 的，也就是能够在 $O(n)$ 时间内初始化。事实上，本文中的结构甚至可以在常数时间内初始化。

Randomized dictionaries are often also referred to as *hash tables*.¹ A hash table is said to have *failure probability* ϵ if each operation takes constant time with probability at least $1 - \epsilon$, and is said to *succeed with high probability* if $\epsilon \leq 1/\text{poly } n$.

Randomized dictionary 通常也被称为 *hash table*。² 如果一个 hash table 的每次操作都以至少 $1 - \epsilon$ 的概率在常数时间内完成，我们就说它的 *failure probability* 为 ϵ ；若 $\epsilon \leq 1/\text{poly } n$ ，则称其 *succeed with high probability*。

A central open question is whether there exists a deterministic constant-time dictionary. A remarkable success in this direction is Pătraşcu and Thorup’s dynamic fusion node [30], which builds on older work by Fredman and Willard [15] in order to construct a deterministic constant-time dictionary for very small sets of keys—that is, sets S of at most $\text{polylog } n$ keys that are $\Theta(\log n)$ bits each. For sets of $\Theta(n)$ keys, it is widely believed that (even non-explicit) deterministic constant-time dictionaries are impossible [37], but we are still very far from having lower bounds to establish this (see [8, 18, 27, 33, 36] for other related work on this question).

一个核心 open question 是：是否存在 deterministic constant-time dictionary。朝这个方向的一个重要成果是 Pătraşcu 和 Thorup 的 dynamic fusion node [30]。它建立在 Fredman 和 Willard [15] 的更早工作之上，能够为很小规模的 key 集合构造 deterministic constant-time dictionary，也就是当 S 的规模最多只有 $\text{polylog } n$ 且每个 key 长度为 $\Theta(\log n)$ 比特时可以做到。对于规模达到 $\Theta(n)$ 的集合，人们普遍相信 deterministic constant-time dictionary 是不可能的，哪怕不要求 explicit [37]；不过我们距离真正证明这一点所需的 lower bound 还差得很远。相关工作可见 [8, 18, 27, 33, 36]。

In this paper, we consider a natural relaxation of this question: What is the smallest failure probability that a hash table can offer [3, 16, 17]? We present the first hash table to achieve a significantly sub-polynomial failure probability. And we show that such a hash table can even be

¹Hash tables are sometimes also informally defined as any solution to the dictionary problem that makes use of hash functions. We intentionally take a more open-ended perspective as to the definition of a hash table, so that we include data structures that accomplish the same goal as traditionally accomplished by hash tables, but using different means.

²有时人们也把 hash table 非正式地定义为“任何使用 hash function 解决 dictionary 问题的方案”。本文刻意采用更宽泛的定义，因此也把那些虽然不直接使用 hash function、但实现了传统 hash table 同样目标的数据结构纳入讨论范围。

made *succinct*, meaning that it uses space within a $(1 + o(1))$ factor of the information-theoretic optimum.

本文考虑这个问题的一个自然放宽版本：hash table 所能达到的最小 failure probability 是多少 [3, 16, 17]？我们给出了第一个真正实现显著次多项式 failure probability 的 hash table，并进一步证明这样的 hash table 甚至还能做成 *succinct* 的，也就是说其空间只比 information-theoretic optimum 多一个 $(1 + o(1))$ 因子。

Past work on super-high-probability guarantees. The study of probabilistic guarantees for hash tables has, up until now, been intimately tied to the study of hash-function families [7, 9, 11, 19, 21, 23–26, 29, 34, 35]. If one has access to fully-random hash functions, then it is known [3, 16, 17] how to achieve substantially sub-polynomial failure probabilities. However, as observed by Goodrich, Hirschberg, Mitzenmacher, and Thaler [17], the known techniques for simulating constant-time hash functions with high independence [11, 26, 35] are *themselves* randomized constructions that introduce an additional $1/\text{poly}(n)$ probability of failure. Efforts at reducing these failure probabilities [17] have only been able to do so at the cost of $\omega(1)$ evaluation times.

到目前为止，hash table 的概率保证研究一直与 hash-function family 的研究密切绑定 [7, 9, 11, 19, 21, 23–26, 29, 34, 35]。如果允许使用 fully-random hash functions，那么已有工作表明可以做到显著次多项式的 failure probability [3, 16, 17]。但是，正如 Goodrich、Hirschberg、Mitzenmacher 和 Thaler [17] 指出的那样，那些把“高独立性 + 常数时间求值”结合起来的已知模拟技术 [11, 26, 35] 本身也是 randomized construction，因此会额外引入 $1/\text{poly}(n)$ 的失败概率。后来虽然有人尝试继续压低这部分失败概率 [17]，但代价都是把求值时间升到 $\omega(1)$ 。

Of the known families of constant-time hash functions, the only one that has been successfully used to obtain a hash table with sub-polynomial failure probabilities is tabulation hashing [29]. Although the standard analyses of tabulation hashing include a $1/\text{poly}(n)$ failure probability, it has been noted by [29] that, in some parameter regimes, the true failure probability is actually sub-polynomial. Indeed, one can extend the techniques of [29] to show that tabulation hash functions are load-balancing with probability $1 - 1/2^{\text{polylog } n}$, thereby allowing one to construct a hash table that has a failure probability of $1/2^{\text{polylog } n}$. To the best of our knowledge, this remains the smallest failure probability to be achieved by any hash table.

在已知的 constant-time hash-function family 中，真正成功用于构造“failure probability 次多项式”的 hash table 的，只有 tabulation hashing [29]。虽然 tabulation hashing 的标准分析通常只给出 $1/\text{poly}(n)$ 的 failure probability，但 [29] 指出，在某些参数区间内，真实 failure probability 其实可以做到次多项式。实际上，沿着 [29] 的技术继续推进，可以证明 tabulation hash functions 以 $1 - 1/2^{\text{polylog } n}$ 的概率具备良好的 load-balancing 性质，因此能够构造出 failure probability 为 $1/2^{\text{polylog } n}$ 的 hash table。据我们所知，这仍然是任何 hash table 已达到的最小 failure probability。

This paper: hash tables with nearly optimal failure probabilities. We introduce a simple data structure, which we call the *amplified rotated trie*, that offers a failure probability of $1/n^{1-\epsilon}$ for an arbitrarily small positive constant ϵ of our choice. Barring a deterministic constant-time dictionary, this is the close to the strongest guarantee that one could hope for: if there were to exist a hash table with failure probability $1/n^{\epsilon}$, for some positive constant $\epsilon > 0$, then that would imply the existence of a (non-explicit) deterministic constant-time dictionary. Our result improves significantly over the previous state-of-the-art of $1/2^{\text{polylog } n}$.

本文引入了一种简单的数据结构，称为 *amplified rotated trie*。它可以把 failure probability 压到 $1/n^{1-\epsilon}$ ，其中 ϵ 是任意小的正常数。除非 deterministic constant-time dictionary 本身存在，否则这已经非常接近人们所能期待的最强保证：因为一旦存在某个 hash table，其 failure probability 达到 $1/n^{\epsilon n}$ ($\epsilon > 0$ 为常数)，就会推出一个非显式的 deterministic constant-time dictionary 的存在。相较于此前最好的 $1/2^{\text{polylog } n}$ ，我们的结果是一次显著提升。

Our second result is that, with a few small modifications, the same data structure can be used to obtain a very different guarantee. The resulting hash table, which we call the *budget rotated trie*, uses $\tilde{O}(\log n)$ random bits to support constant-time operations with high probability in n . This guarantee, which has also been achieved using more classical hashing-based techniques in previous work by Dietzfelbinger et al. [10], serves as a natural dual to the one above — rather than trying to minimize failure probability, while using up to $O(n)$ random bits, one tries to minimize random bits while maintaining a standard $1/\text{poly}(n)$ failure probability.

我们的第二个结果展示了另一种完全不同的权衡：只需做少量修改，同一类数据结构就能得到另一种保证。所得的 hash table 我们称为 *budget rotated trie*。它只使用 $\tilde{O}(\log n)$ 个 random bits，同时仍能以 high probability 在常数时间内支持各项操作。这一保证在早先 Dietzfelbinger 等人的更经典 hashing 方案中也已出现过 [10]，但它与上面的结果形成了一个很自然的对偶：上面的目标是在允许使用最多 $O(n)$ random bits 的前提下尽可能压低 failure probability，而这里的目标则是在维持标准 $1/\text{poly}(n)$ failure probability 的同时，尽可能减少 random bits。

An interesting feature of budget rotated tries is that they are able to make use of so called “gradually-increasing-independence hash functions” [7, 23]. These hash functions, introduced originally by Celis, Reingold, Segev, and Wieder [7] (and subsequently made more efficient by Meka, Reingold, Rothblum, and Rothblum [23]) can be used to distribute n balls roughly evenly across n bins using only $O(n \log \log n)$ random bits, but come with the seemingly significant drawback that they require $\Theta((\log \log n)^2)$ time to evaluate. As a consequence, past work on applying these hash functions to classical hash tables [32] has incurred $\omega(1)$ time per operation. Our approach suggests that such gradually-increasing-independence may be more broadly applicable to than was previously thought, and can be used in the design of constant-time data structures.

budget rotated trie 的一个有趣特征，是它能够利用所谓的 “gradually-increasing-independence hash functions” [7, 23]。这类 hash functions 最初由 Celis、Reingold、Segev 和 Wieder [7] 提出，之后又被 Meka、Reingold、Rothblum 和 Rothblum [23] 做了效率改进。它们可以只用 $O(n \log \log n)$ 个 random bits，把 n 个 balls 大致均匀地分配到 n 个 bins 中；但其看似严重的缺点是，求值时间需要 $\Theta((\log \log n)^2)$ 。因此，以往把它们用于经典 hash table 的工作 [32] 都不得不接受每次操作 $\omega(1)$ 的时间。我们的做法表明，这种 gradually-increasing-independence 的适用范围其实可能比过去想象的更广，也完全可以进入 constant-time data structure 的设计中。

Achieving succinctness. Finally, we turn our attention to space efficiency. There has also been a great deal of work on how to construct a *succinct hash table* (see, e.g., [3, 4, 20, 31]), that is, a hash table that stores n key/values pairs from a universe U in space

$$(1 + o(1))\mathcal{B}(|U|, n)$$

bits, where $\mathcal{B}(|U|, n) = \log \binom{|U|}{n}$ is the information-theoretic lower bound on the size of any hash table.

最后，我们转向空间效率问题。关于如何构造 *succinct hash table*，也已有大量工作（例

如 [3, 4, 20, 31])。所谓 succinct hash table, 是指: 它能把来自 universe U 的 n 个 key/value 对存储在

$$(1 + o(1))\mathcal{B}(|U|, n)$$

比特空间内, 其中 $\mathcal{B}(|U|, n) = \log \binom{|U|}{n}$ 是任意 hash table 大小所能达到的 information-theoretic lower bound。

In Section 6, we show that the data structures in this paper can also be made succinct, in the parameter regime where keys/values are $(1 + \Theta(1)) \log n$ bits. More generally, we give a black-box transformation that can be applied to any dictionary in order to obtain a succinct dictionary whose probabilistic guarantees are nearly the same as the original's. The new dictionary uses $\mathcal{B}(|U|, n) + O(n(\log n)/\log \log n)$ bits.

在第 6 节中, 我们证明: 在 key/value 长度属于 $(1 + \Theta(1)) \log n$ 的参数区间内, 本文给出的数据结构也都可以做成 succinct 的。更一般地, 我们给出一个 black-box transformation, 可以把任意 dictionary 转换成一个 succinct dictionary, 而且其概率保证几乎与原结构相同。转换后的新字典使用 $\mathcal{B}(|U|, n) + O(n(\log n)/\log \log n)$ 比特空间。

Interestingly, the transformation itself makes use of our (non-succinct) budget rotated trie as a critical algorithmic component. The transformation also makes use of a reduction due to Raman and Rao [31], and can be seen as a constant-time and randomness-efficient version of the succinct dictionary given in [31] (which guaranteed only constant expected-time operations).

有意思的是, 这个 transformation 本身把我们前面构造的非 succinct 的 budget rotated trie 当作一个关键算法组件来使用。它同时还利用了 Raman 和 Rao [31] 的一个 reduction, 因此也可以看作是 [31] 中 succinct dictionary 的一个“常数时间 + 随机位高效”版本, 而后者当时只保证 constant expected-time operations。

Applying our transformation, we obtain two data structures: we get a succinct hash table that uses $O(\log n(\log \log n)^3) = \tilde{O}(\log n)$ random bits, while supporting constant-time operations with high probability; and a succinct hash table with a failure probability of $1/n^{1-\epsilon}$, where ϵ is an arbitrarily small positive constant of our choice.

把这个 transformation 应用于前文结构之后, 我们得到两种 succinct 数据结构: 一种 succinct hash table 只使用 $O(\log n(\log \log n)^3) = \tilde{O}(\log n)$ 个 random bits, 同时以 high probability 支持常数时间操作; 另一种 succinct hash table 的 failure probability 则达到 $1/n^{1-\epsilon}$, 其中 ϵ 可以任意小。

Circumventing the hash-function bottleneck. At the core of our results is a simple but powerful observation: that it is possible to construct a hash table *that does not use hash functions*, and that is consequently free of the limitations that hamper known hash-function constructions. In particular, we begin our exposition by constructing a simple randomized dictionary that we call a *rotated radix trie*. Like standard hash tables, the rotated radix trie uses linear space and is constant-time (with high probability). But unlike standard hash tables, which rely on randomness supplied by hash functions, the rotated radix trie uses randomness directly embedded into the data structure. The rotated radix trie then serves as the basis for both the amplified rotated trie and the budget rotated trie.

本文所有结果的核心，是一个简单但有力的观察：完全可能构造一种不使用 *hash functions* 的 hash table，因此也就能绕开目前已知 hash-function constructions 所面临的各种限制。具体来说，我们的叙述从一个简单的 randomized dictionary 开始，它被称为 *rotated radix trie*。和标准 hash table 一样，它使用线性空间，并且以 high probability 支持常数时间操作；但不同于标准 hash table 依赖 hash functions 提供随机性，rotated radix trie 的随机性是直接嵌入到数据结构内部的。随后，无论是 amplified rotated trie 还是 budget rotated trie，都是建立在它之上的。

Outline. The rest of the paper proceeds as follows. Section 2 presents basic preliminaries and conventions—these are specialized for the setting of non-succinct dictionaries, and slightly different conventions are established later on in Section 6 for discussing succinct dictionaries. Section 3 presents and analyzes the rotated radix trie. Building on this, Section 4 gives a hash table that achieves failure probability $1/n^{1-\epsilon}$ and Section 5 gives a high-probability hash table using $O(\log n \log \log n)$ random bits; in Appendix A, we show that the latter guarantee can also be extended to the case where machine words are $\omega(\log n)$ bits. Finally, in Section 6, we show how to transform any linear-space hash table into a succinct hash table, while nearly preserving the randomization guarantees of the data structure.

本文余下部分安排如下。第 2 节给出基础 preliminaries 和 conventions，它们是专门为 non-succinct dictionary 这一语境准备的；到第 6 节讨论 succinct dictionaries 时，我们会再换用一套稍有不同的 conventions。第 3 节介绍并分析 rotated radix trie。在此基础上，第 4 节给出 failure probability 达到 $1/n^{1-\epsilon}$ 的 hash table，第 5 节给出只使用 $O(\log n \log \log n)$ random bits 的 high-probability hash table；在附录 A 中，我们还说明后一种保证可以推广到 machine word 长度为 $\omega(\log n)$ 的情形。最后，在第 6 节中，我们展示如何把任意线性空间 hash table 转换成 succinct hash table，同时几乎保留其随机化保证。

2 Preliminaries and Conventions (for Non-Succinct Dictionaries)

We now present several preliminary definitions and conventions for discussing (non-succinct) dictionaries. These conventions are used throughout the paper, except in Section 6 where we consider succinct dictionaries. We end up using slightly different conventions when discussing non-succinct versus succinct dictionaries because, in the non-succinct setting, there are a number of standard simplifications that one can make without loss of generality (but which do not hold in the succinct setting).

下面我们给出讨论 non-succinct dictionaries 时所需的一些预备定义与约定。除第 6 节外，这些约定贯穿全文；而在 succinct dictionary 的语境中，我们会换用一套稍有不同的 conventions。原因在于：在 non-succinct setting 下，有一些标准化简可以在不失一般性的前提下使用，但这些化简在 succinct setting 中并不成立。

Keys, values, and dictionaries. Let $U = [\text{poly}(n)]$ be the set of all possible $\Theta(\log n)$ -bit keys, and let $V = [\text{poly}(n)]$ be the set of all possible $\Theta(\log n)$ -bit values. A *dictionary* is a data structure that stores a set of keys from U , and that associates each key x with a value $y \in V$. Dictionaries support three operations: $\text{Insert}(x, y)$ adds key x to the set, if it is not already there, and sets the corresponding value to y ; $\text{Delete}(x)$ removes x ; and $\text{Query}(x)$ reports whether key x is present, returning the corresponding value if so.

设 $U = [\text{poly}(n)]$ 为所有可能的 $\Theta(\log n)$ 比特 key 的集合, $V = [\text{poly}(n)]$ 为所有可能的 $\Theta(\log n)$ 比特 value 的集合。一个 **dictionary** 是这样一种数据结构: 它维护来自 U 的一组 keys, 并为每个 key x 关联一个 value $y \in V$ 。dictionary 支持三种操作: $\text{Insert}(x, y)$ 在 x 尚不存在时把它加入集合, 并把其对应 value 设为 y ; $\text{Delete}(x)$ 删除 x ; $\text{Query}(x)$ 判断 x 是否存在, 若存在则返回相应 value。

When discussing non-succinct dictionaries, we focus (without loss of generality) on fixed-capacity dictionaries, that is, dictionaries that are permitted to have up to n keys at a time. Such dictionaries can be used to implement dynamically-resized dictionaries by simply rebuilding the dictionary (in a deamortized fashion) whenever its size changes by a constant factor. Unless stated otherwise, we shall require implicitly that dictionaries must use at most linear space (i.e., $O(n \log n)$ bits) and have $O(1)$ initialization time.

在讨论 non-succinct dictionary 时, 我们不失一般性地只关注 fixed-capacity dictionary, 也就是任意时刻最多容纳 n 个 key 的字典。这样的结构可以通过在规模变化达到常数倍时重建字典 (并将其去摊还化) 来实现动态扩缩容。除非另行说明, 本文默认 dictionary 只使用线性空间, 即 $O(n \log n)$ 比特, 并且具有 $O(1)$ 的初始化时间。

Standard techniques for simplifying dictionaries. There are several standard reductions that can be used to simplify the problem of maintaining a linear-space dictionary.

有几种标准 reduction 可以用来简化“维护线性空间 dictionary”这一问题。

We can assume without loss of generality that the lifespan of a dictionary is only $O(n)$ operations. Indeed, longer sequences of operations can be broken into phases of size $O(n)$, and the dictionary can be rebuilt from scratch during each phase (i.e., all of the elements are gradually moved from one instance of the dictionary to another new instance of the dictionary). The rebuild cost can be spread across the phase so that the asymptotic running times of operations are preserved.³

首先, 我们可以不失一般性地假设一个 dictionary 的生命周期只有 $O(n)$ 次操作。原因是, 更长的操作序列可以被拆成若干个大小为 $O(n)$ 的 phase, 并在每个 phase 中从头重建字典, 也就是把所有元素逐步从旧实例迁移到新实例。重建代价可以均摊到整个 phase 上, 因此不会改变单次操作的渐近运行时间。⁴

Since the lifespan of each phase is only $O(n)$ operations, we can implement deletions with the following trivial approach: simply mark elements as deleted, and defer the actual removal of those elements until the next rebuild. As a consequence, when designing the dictionary that will be used to implement each phase, we can assume without loss of generality that the only operations performed are insertions/queries.

由于每个 phase 的生命周期只有 $O(n)$ 次操作, 我们还可以用一个非常直接的方法处理 deletion: 只需把元素标记为 deleted, 而把真正的物理删除推迟到下次 rebuild 再做。因此, 在为单个 phase 设计 dictionary 时, 我们不失一般性地可以假设真正发生的操作只有 insertions 和 queries。

³For our purposes, rebuilds *do not* sample new random bits. Once a dictionary's random bits are chosen, they are fixed forever.

⁴对本文而言, rebuild 不会重新抽取 random bits。一旦某个 dictionary 的 random bits 被选定, 它们就永远固定不变。

We will therefore assume throughout the paper that, whenever we are discussing a non-succinct dictionary, the sequence of operations being performed has length $O(n)$ and consists exclusively of inserts/queries.

因此，全文中凡是讨论 non-succinct dictionary 的地方，我们都默认操作序列长度为 $O(n)$ ，并且只包含 inserts/queries。

Randomization. Randomized dictionaries are given access to a stream of random bits—the dictionary can access the next $\Theta(\log n)$ bits of the stream in time $O(1)$. When analyzing a randomized dictionary, the goal is to bound the **failure probability** for any given operation. We emphasize that, in this context, failure does not refer to lack of correctness, but instead to lack of timeliness. A dictionary **fails** whenever an operation takes super-constant time. (Later, in Section 6, when we consider succinct dictionaries, we will also allow for failures with respect to space consumption.)

Randomized dictionary 可以访问一串 random bits，也就是它能够在 $O(1)$ 时间内读取接下来的 $\Theta(\log n)$ 个随机位。分析 randomized dictionary 时，我们关注的是任意单次操作的 **failure probability**。需要强调的是，这里的 failure 并不意味着结果错误，而是意味着“不够及时”：只要某次操作花费了超常数时间，我们就认为 dictionary **fails**。到了第 6 节讨论 succinct dictionary 时，我们还会把空间消耗超标也视为 failure。

All of our dictionaries share the property that, once a failure occurs, all of the rest of the operations (in the current phase of $O(n)$ inserts/queries) also fail. We will not bother to explicitly specify the dictionary’s behavior when a failure occurs, since at that point it is okay for each of the remaining operations in the phase to take linear time.

本文中的所有 dictionary 都具有同一种性质：一旦某次 failure 发生，那么当前这个由 $O(n)$ 次 inserts/queries 组成的 phase 里剩余的所有操作也都会失败。我们不会再细致规定 failure 发生后的具体行为，因为从那一刻开始，余下每一步即便都退化到线性时间也是允许的。

We remark that randomized data structures are analyzed against *oblivious* adversaries, meaning that the sequence of insertions/deletions/queries being performed is determined independently of the random bits that the dictionary uses. We also remark that the failure probability of a dictionary is determined on a per-operation basis. For example, if a dictionary has failure-probability p and is used for $1/p$ operations, then it is reasonable that some failures should occur.⁵

我们还说明两点。第一，randomized data structure 默认在 *oblivious* adversary 模型下分析，也就是说操作序列是在不知道 dictionary 所用 random bits 的前提下事先决定的。第二，dictionary 的 failure probability 是按“每次操作”来定义的。例如，如果某个 dictionary 的单次 failure probability 为 p ，而它被连续使用了 $1/p$ 次，那么出现一些 failures 本来就是合理的。⁶

⁵Moreover, failures may be correlated between steps (and between phases). For example, if we are using r random bits, and an adversary guesses them, then they can force failures all the time with probability $1/2^r$.

⁶此外，failure 在不同步骤之间、甚至不同 phase 之间都可能相关。例如如果结构使用了 r 个 random bits，而 adversary 恰好猜中了它们，那么 adversary 就能以 $1/2^r$ 的概率一直强行触发 failure。

3 A Warmup Data Structure: The Rotated Trie.

In this section, we present a simple randomized constant-time dictionary, called the *rotated radix trie*, that serves as the basis for the data structures in later sections.

本节给出一个简单的 randomized constant-time dictionary, 称为 *rotated radix trie*。它将作为后续所有数据结构构造的基础。

The starting place: an n -ary radix trie. The starting place for our data structure will be the classic n -ary radix trie. Each internal node of the trie can be viewed as an array of size n , where the j -th entry of the array stores either a pointer to child j , if such a child exists, or a null character otherwise. The leaves of the trie correspond to the keys in the data structure (and are where we store values). In general, there is a leaf with root-to-leaf path $j_1, j_2, j_3, \dots, j_d$ if and only if the key $j_1 \circ j_2 \circ j_3 \circ \dots \circ j_d \in [n^d] = [U]$ is present.

我们的起点是经典的 n 叉 radix trie。trie 的每个 internal node 都可以看成一个大小为 n 的数组, 其中第 j 个位置要么保存指向 child j 的指针, 要么为空。trie 的叶子对应数据结构中实际存储的 keys, 同时也在叶子处存放 values。一般来说, 当且仅当 key $j_1 \circ j_2 \circ \dots \circ j_d \in [n^d] = [U]$ 存在时, trie 中才会出现一条 root-to-leaf path 为 $j_1, j_2, \dots, j_d$ 的叶子。

What makes the n -ary radix trie an interesting starting place is that the trie deterministically supports constant-time operations. What it does not support is space efficiency: there may be as many as $\Theta(n)$ internal nodes, each of which is an array of size n , and which collectively require space $\Omega(n^2)$ to implement.

n 叉 radix trie 之所以是一个有趣的起点, 是因为它可以 deterministic 地支持常数时间操作。它的不足在于空间效率极差: 可能会出现多达 $\Theta(n)$ 个 internal nodes, 而每个节点又都是一个大小为 n 的数组, 因此总空间会达到 $\Omega(n^2)$ 。

Using randomness to save space: the rotated radix trie. We now add randomness to our data structure in a very simple way. Label the internal nodes of the trie by $1, 2, \dots, m$ for some $m \in O(n)$, and refer to the array used to implement each internal node $i \in [m]$ as A_i . When the data structure is initialized, we assign to each internal node i a *random rotation* r_i selected uniformly at random from $\{0, 1, \dots, n-1\}$. The rotation r_i is stored as part of the node i .

现在我们以一种非常简单的方式向这个数据结构中加入随机性。把 trie 的 internal nodes 编号为 $1, 2, \dots, m$, 其中 $m \in O(n)$, 并把实现第 i 个 internal node 的数组记作 A_i 。当数据结构初始化时, 我们为每个 internal node i 指派一个 *random rotation* r_i , 它从 $\{0, 1, \dots, n-1\}$ 中均匀随机选取, 并作为该 node 状态的一部分被保存下来。

The purpose of r_i is to apply a random cyclic rotation to the array A_i . That is, if a pointer would have been stored in position j of A_i , it is now stored in position $((j + r_i) \bmod n)$ of A_i instead.

r_i 的作用, 是给数组 A_i 施加一个随机循环位移。也就是说, 原本应该存放在 A_i 第 j 个位置的指针, 现在改为存放在第 $((j + r_i) \bmod n)$ 个位置。

Finally, having rotated each of the arrays A_i by r_i , we now overlay the arrays $A_1, A_2, \dots, A_m$

on top of one another, and we store the contents of all of them in a single array A of size n . Of course, the j -th position of A may be responsible for storing elements from multiple A_i s. As long as the number of elements stored in each entry is relatively small, then this is fine: we simply implement each entry of A as a dynamic fusion node [30], which is a deterministic constant-time linear-space dictionary capable of storing up to $\ell = \text{polylog } n$ key/value pairs at a time.

接着, 在把每个 A_i 按照 r_i 旋转之后, 我们把 $A_1, A_2, \dots, A_m$ 叠在一起, 并把它们的所有内容压缩存放到一个大小为 n 的统一数组 A 中。显然, A 的第 j 个位置可能同时承担来自多个 A_i 的元素。只要每个位置承载的元素数目不大, 这没有问题: 我们只需把 A 的每个位置实现成一个 dynamic fusion node [30], 它是一个 deterministic、constant-time、linear-space 的 dictionary, 可以容纳最多 $\ell = \text{polylog } n$ 个 key/value 对。

If, prior to collapsing the arrays into a single array A , the the j -th position of rotated array A_i stored a pointer to array $A_{i'}$, then afterwards the dynamic fusion node $A[j]$ stores the key-value pair $(i, (i', r_{i'}))$. In this setting, we refer to the pair $(i', r_{i'})$ as a **pointer** to $A_{i'}$, since it dictates which array $A_{i'}$ we are pointing at and where to find the entries of $A_{i'}$. Similarly, if prior to collapsing the arrays, the j -th position of the rotated array A_i stored a pointer p directly to a value (rather than to another array $A_{i'}$), then the dynamic fusion node $A[j]$ stores the key-value pair (i, p) .

更具体地说, 如果在把各数组压缩到统一数组 A 之前, 旋转后的 A_i 的第 j 个位置本来存的是指向数组 $A_{i'}$ 的指针, 那么压缩之后, dynamic fusion node $A[j]$ 中就保存键值对 $(i, (i', r_{i'}))$ 。这里我们把 $(i', r_{i'})$ 称为指向 $A_{i'}$ 的 **pointer**, 因为它既说明了要跳转到哪一个数组, 又说明了如何在那个数组中找到其条目。类似地, 如果旋转后的 A_i 的第 j 个位置原本直接存放的是指向某个 value 的指针 p , 而不是指向另一个数组 $A_{i'}$, 那么压缩后 $A[j]$ 就保存键值对 (i, p) 。

Analyzing the rotated radix trie. To analyze the rotated radix trie, we must show that, with high probability in n , each entry of A is responsible for storing entries from at most $\ell = \text{polylog } n$ different A_i s.

分析 rotated radix trie 的关键, 是证明: 以关于 n 的 high probability, 数组 A 的每个位置至多只需要承载来自 $\ell = \text{polylog } n$ 个不同 A_i 的条目。

Let us first establish some conventions that will be useful throughout the rest of the paper. When discussing a radix trie, we will refer to the arrays $A_1, A_2, \dots, A_m$ as the **nodes** (or sometimes as the **internal nodes**), and we will refer to the non-null entries of each A_i (i.e., the entries containing pointers) as **balls**.

先引入一些在后文中都会用到的记号。讨论 radix trie 时, 我们把数组 $A_1, A_2, \dots, A_m$ 称为 **nodes**, 有时也叫 **internal nodes**; 而每个 A_i 的非空条目, 也就是那些保存指针的位置, 则被称为 **balls**。

In total, there are $O(n)$ balls in the trie. Each ball b is specified by a pair $(s, c) \in [m] \times [n]$, where $s \in [m]$ is the **source node** for the ball (i.e., the node containing the ball), and $c \in [n]$ is the **child index** of the ball (i.e., the index in A_i where b is logically stored). The effect of randomly rotating the arrays A_i and then overlaying them to obtain a single array A is that each ball $b = (s, c)$ gets mapped to position $\phi(s, c) := c + r_s$ in A . We refer to the entries of A as **bins**, so each ball b gets mapped to bin $\phi(b)$. The dynamic fusion node for a each bin $j \in [n]$ stores the set of key-value pairs (b, p) where b ranges over the balls satisfying $\phi(b) = j$, and p is the pointer corresponding to the ball b .

整个 trie 中总共有 $O(n)$ 个 balls。每个 ball b 都可以表示为一对 $(s, c) \in [m] \times [n]$, 其中 $s \in [m]$ 是该 ball 的 *source node*, 也就是包含它的那个 node; 而 $c \in [n]$ 是该 ball 的 *child index*, 也就是它在逻辑上应位于 A_i 中的哪个位置。对所有数组 A_i 做随机旋转并叠加为一个统一数组 A 的效果, 就是把每个 ball $b = (s, c)$ 映射到 A 中的位置 $\phi(s, c) := c + r_s$ 。我们把 A 的各个位置称为 *bins*, 因此每个 ball b 会被映射到 bin $\phi(b)$ 。对于任意 bin $j \in [n]$, 其对应的 dynamic fusion node 存储了所有满足 $\phi(b) = j$ 的 balls 所对应的键值对 (b, p) , 其中 p 是该 ball 所携带的 pointer。

For $i \in [m]$ and $j \in [n]$, let $X_{i,j}$ be the 0-1 random variable indicating whether node i places a ball into bin j . The $X_{i,j}$ s are not independent across the bins j , but they are independent across the nodes i , since each $X_{i,j}$ is a function of the random bits r_i . Therefore, the number Y_j of balls in bin j , which is given by $Y_j = \sum_{i=1}^m X_{i,j}$, is a sum of independent indicator random variables.

对任意 $i \in [m]$ 和 $j \in [n]$, 令 $X_{i,j}$ 表示一个 0-1 随机变量, 它指示 node i 是否向 bin j 放入了一个 ball。虽然对不同的 bins j 来说, 这些 $X_{i,j}$ 并不独立; 但他们在不同的 nodes i 之间是独立的, 因为每个 $X_{i,j}$ 都只是 random bits r_i 的函数。因此, bin j 中 balls 的总数

$$Y_j = \sum_{i=1}^m X_{i,j}$$

就是一组相互独立的指示变量之和。

Each of the $O(n)$ balls has probability $1/n$ of being in bin j , so $\mathbb{E}[Y_j] = O(1)$. Thus, by a Chernoff bound, we have that $Y_j \leq \text{polylog } n$ with high probability in n . The Chernoff bound actually tells us that $Y_j \leq \text{polylog } n$ with probability $1/n^{\text{polylog } n}$, so we have even achieved a slightly sub-polynomial probability of failure.

由于总共有 $O(n)$ 个 balls, 而且每个 ball 落到某个固定 bin j 的概率都是 $1/n$, 因此 $\mathbb{E}[Y_j] = O(1)$ 。于是由 Chernoff bound 可以得到: 以 high probability, $Y_j \leq \text{polylog } n$ 。事实上, Chernoff bound 还能给出更强的结论, 即 $Y_j \leq \text{polylog } n$ 失败的概率只有 $1/n^{\text{polylog } n}$, 因此这里已经实现了一种“略微次多项式”的 failure probability。

Putting the pieces together. If we implement deletions as in Section 2, then we obtain the following result:

如果按第 2 节所说的方式处理 deletions, 就得到如下结论:

Proposition 1. *The rotated radix trie is a randomized linear-space dictionary that can store up to $n \Theta(\log n)$ -bit keys/values at a time, and that supports each operation in constant time with probability $1 - 1/n^{\text{polylog } n}$.*

也就是说, rotated radix trie 是一个 randomized、linear-space 的 dictionary。它可以同时容纳最多 n 个 $\Theta(\log n)$ 比特的 key/value 对, 并且每次操作都以 $1 - 1/n^{\text{polylog } n}$ 的概率在常数时间内完成。

It's worth taking a moment to remark on how to initialize our data structure. The random rotations r_i can be initialized lazily, so that r_i is generated the first time that the node i is used. Additionally, we do not have to actually pay the cost of initializing any arrays, since we can use

standard techniques to simulate zero-initialized arrays in constant time (see [1] or Problem 9 of Section 1.6 of [5]). Thus our rotated radix trie can be initialized in constant time.

这里还值得额外说明一下初始化问题。各个随机 rotation r_i 可以懒初始化，也就是说在 node i 第一次被用到时才生成对应的 r_i 。另外，我们也不需要真的花时间去初始化所有数组，因为可以使用标准技巧在常数时间内模拟 zero-initialized arrays (参见 [1] 或 [5] 第 1.6 节习题 9)。因此，rotated radix trie 的整体初始化时间仍然是常数。

Taking stock of our situation. The rotated radix trie does not, on its own, make any significant progress on either of the problems that we care about: (1) achieving super-high probability guarantees, and (2) using a near-logarithmic number of random bits. We have achieved a *slightly* sub-polynomial failure probability, but we are nowhere near our goal of $1/n^{1-\epsilon}$.

不过，单独看 rotated radix trie，它还没有在本文真正关心的两个目标上取得决定性进展：第一是 super-high probability guarantee，第二是接近对数级别的 random bits 使用量。我们现在得到的 failure probability 虽然已经略优于多项式级别，但距离目标 $1/n^{1-\epsilon}$ 还相差很远。

What makes the rotated radix trie useful, however, is that the role of randomness in the data structure is remarkably simple. The *only* sources of randomness are the rotational offsets $r_1, r_2, \dots, r_m$. In this sense, the rotated radix trie deviates from the standard mold for how to design a constant-time dictionary. The randomness in the data structure isn't used to hash elements, but is instead used to apply random rotations to sparse arrays.

尽管如此，rotated radix trie 仍然非常有价值，因为其中随机性的角色异常简单。整个数据结构里 唯一的随机来源就是这些旋转偏移量 $r_1, r_2, \dots, r_m$ 。从这个角度看，rotated radix trie 已经明显偏离了设计 constant-time dictionary 的传统套路：这里的随机性不是用来对元素做 hashing，而是用来对稀疏数组施加随机 rotation。

Since the role of randomness will be important in later sections, we conclude the current section by discussing an important subtlety in how the randomness is re-purposed over time. Consider how the data structure evolves over a large period of time containing many insertions/deletions. As the shape of the trie changes, each array A_i will be repurposed to represent different parts of the trie. This means that the way in which the random rotation r_i interacts with the key space also changes over time, with the same r_i applying to a different node in the trie (and thus a different part of the key space) at different times. The re-purposing of r_i s has an interesting consequence: even if two points in time t_1 and t_2 store the exact same set S of key/value pairs as one-another, the shape of the rotated trie may differ considerably between the two times.

由于随机性的角色在后文会变得非常重要，因此我们在本节最后指出一个关键细节：randomness 会随着时间被“重新用途化”。设想这个数据结构在很长一段时间内经历了大量 insertions/deletions。随着 trie 形状的变化，每个数组 A_i 会被不断重新拿去表示 trie 的不同部分。这意味着随机 rotation r_i 与 key space 的交互方式也会随时间改变：同一个 r_i 在不同时间可能作用于 trie 中不同的 node，也就对应 key space 的不同区域。 r_i 这种重新用途化带来一个有趣后果：即便两个时间点 t_1 和 t_2 存储的是完全相同的一组 key/value 对 S ，它们对应的 rotated trie 形状也可能相当不同。

4 The Amplified Rotated Trie

In this section, we modify the rotated radix trie to reduce its probability of failure (i.e., the probability that a given operation takes super-constant time) to $1/n^{1-\epsilon}$, for a positive constant ϵ of our choice. We will refer to this new data structure as the *amplified rotated radix trie*.

本节中，我们将对 rotated radix trie 做进一步修改，把它的 failure probability，也就是“某次操作花费超常数时间”的概率，降低到 $1/n^{1-\epsilon}$ ，其中 ϵ 是我们任选的一个正常数。我们把得到的新结构称为 *amplified rotated radix trie*。

Storing overflow balls in a (non-rotated) trie. Whenever a ball b is inserted into a bin j that already contains $\ell = \text{polylog } n$ other balls, the ball b is regarded as an *overflow ball*. Since each bin is a dynamic fusion node with capacity ℓ , we cannot store the overflow balls in the bins.

当一个 ball b 被插入到某个 bin j ，而该 bin 中已经有了 $\ell = \text{polylog } n$ 个其他 balls 时，我们就把 b 称为一个 *overflow ball*。由于每个 bin 都是容量只有 ℓ 的 dynamic fusion node，因此这些 overflow balls 无法再继续存放在 bins 中。

We instead store the overflow balls in a secondary data structure Q that is implemented as a n^δ -ary trie, for some positive constant $\delta > 0$.

为此，我们把 overflow balls 存放到一个辅助数据结构 Q 中，它实现为一个 n^δ 叉 trie，其中 $\delta > 0$ 为某个正常数。

The secondary data structure Q supports inserts/queries on overflow balls in constant time. On the other hand, Q is not space efficient. If there are q overflow balls, then Q may use as much as qn^δ space. To establish that our dictionary uses linear space, we must show that

辅助结构 Q 能够在常数时间内支持对 overflow balls 的 insert/query 操作；但另一方面，它的空间效率并不好。如果当前有 q 个 overflow balls，那么 Q 的空间可能高达 qn^δ 。因此，为了证明整个 dictionary 仍然只用线性空间，我们必须证明

$$\Pr[q \geq n^{1-\delta}] \leq O\left(1/n^{1-\epsilon}\right). \quad (1)$$

The problem: dependencies between balls with shared source nodes. Our current data structure does not yet satisfy (1), however. This is because, whenever multiple balls share the same source node, their assignments become closely linked. Suppose, for example, that the rotated trie R has only 2ℓ internal nodes, and that each internal node $i \in \{1, 2, \dots, 2\ell\}$ contains $\Theta(n/\ell)$ balls $(i, 1), (i, 2), \dots, (i, \Theta(n/\ell))$. With probability $1/n^{2\ell} = 1/2^{\text{polylog } n}$, each internal node $i \in \{1, 2, \dots, 2\ell\}$ has random rotation $r_i = 0$. This results in bins $1, 2, \dots, \Theta(n/\ell)$ each containing 2ℓ balls—in other words, half of the balls in the system are overflow balls. This means that

$$\Pr[q \geq \Omega(n)] \geq 1/2^{\text{polylog } n}.$$

That is, our failure probability using Q to store overflow balls is *no better* than the failure probability that we achieved in Section 3 without Q .

不过, 当前这个数据结构还不能满足式 (1)。问题在于: 只要多个 balls 共享同一个 source node, 它们的落点就会彼此强相关。举个例子, 假设 rotated trie R 只有 2ℓ 个 internal nodes, 并且每个 internal node $i \in \{1, 2, \dots, 2\ell\}$ 都包含 $\Theta(n/\ell)$ 个 balls $(i, 1), (i, 2), \dots, (i, \Theta(n/\ell))$ 。以概率 $1/n^{2\ell} = 1/2^{\text{polylog } n}$, 这些 internal nodes 的 random rotation 全都恰好满足 $r_i = 0$ 。这时 bins $1, 2, \dots, \Theta(n/\ell)$ 中的每一个都会装下 2ℓ 个 balls, 也就是说系统中大约有一半 balls 都会成为 overflow balls。因此

$$\Pr[q \geq \Omega(n)] \geq 1/2^{\text{polylog } n}.$$

换句话说, 即便我们引入了 Q 来存储 overflow balls, 它所带来的 failure probability 也并不比第 3 节中不使用 Q 时的 failure probability 更好。

Reducing the dependencies. What makes the above pathological example possible is that it is possible to have only a small number of internal nodes in our rotated trie. This makes it so that there are only a small number of random bits that affect the rotated trie's structure, preventing us from achieving any super-high probability guarantees.

上面那个病态例子之所以成立, 根本原因在于 rotated trie 中 internal nodes 的数量可能过少。这样一来, 真正影响 trie 结构的 random bits 数量也会过少, 从而无法支撑任何 super-high probability guarantee。

To fix this problem, we reduce the fanout of our rotated radix trie from n to n^δ . Now each internal node can contain at most n^δ balls, so there are guaranteed to be at least $n^{1-\delta}$ internal nodes. This ensures that there are always at least $n^{1-\delta} \log n$ random bits affecting the trie's structure.

为解决这个问题, 我们把 rotated radix trie 的 fanout 从 n 降到 n^δ 。此时每个 internal node 最多只能包含 n^δ 个 balls, 因此整个 trie 至少会拥有 $n^{1-\delta}$ 个 internal nodes。这样就能保证: 始终至少有 $n^{1-\delta} \log n$ 个 random bits 共同影响 trie 的结构。

We remark that, since the fanout of the rotated trie is now n^δ , each ball is determined by a pair (s, c) where $s \in [m]$ is a source node and $c \in [n^\delta]$ is a child index. Nonetheless, the mapping ϕ from balls to bins works exactly as before: we map ball (s, c) to bin $\phi(s, c) = ((r_s + c) \bmod n)$ where $r_s \in [n]$ is selected at random.

需要注意的是, 由于现在 rotated trie 的 fanout 变成了 n^δ , 每个 ball 都由一对 (s, c) 决定, 其中 $s \in [m]$ 是 source node, $c \in [n^\delta]$ 是 child index。不过, balls 到 bins 的映射 ϕ 本身并没有变化: 我们仍把 ball (s, c) 映射到 bin $\phi(s, c) = ((r_s + c) \bmod n)$, 其中 $r_s \in [n]$ 是随机选取的。

Bounding the number of overflow balls. Of course, there are still dependencies between the number q_j of overflow balls in different bins $j \in [n]$. To handle these dependencies, we make use of a tool from probabilistic combinatorics.

当然, 不同 bins $j \in [n]$ 中 overflow balls 的数量 q_j 之间仍然存在依赖。为了处理这些依赖, 我们引入概率组合学中的一个工具。

Call a function $f : [0, 1]^m \rightarrow \mathbb{R}$ **L -Lipschitz** if for every pair of inputs of the form $\vec{x} = (x_1, \dots, x_i, \dots, x_m)$ and $\vec{x}' = (x_1, \dots, x'_i, \dots, x_m)$, we have $|f(\vec{x}) - f(\vec{x}')| \leq L$. McDiarmid's in-

equality [22] tells us that if f is L -Lipschitz and $X_1, X_2, \dots, X_m \in [0, 1)$ are independent random variables, then for any $t \geq 0$,

$$\Pr[|f(X_1, \dots, X_m) - \mathbb{E}[f(X_1, \dots, X_m)]| \geq t] \leq 2e^{-2t^2/(mL^2)}.$$

如果一个函数 $f : [0, 1)^m \rightarrow \mathbb{R}$ 满足如下性质: 对任何只在第 i 个坐标不同的一对输入 $\vec{x} = (x_1, \dots, x_i, \dots, x_m)$ 与 $\vec{x}' = (x_1, \dots, x'_i, \dots, x_m)$, 都有 $|f(\vec{x}) - f(\vec{x}')| \leq L$, 那么我们就称 f 是 **L -Lipschitz** 的。McDiarmid 不等式 [22] 告诉我们: 如果 f 是 L -Lipschitz 的, 而 $X_1, X_2, \dots, X_m \in [0, 1)$ 是独立随机变量, 那么对任意 $t \geq 0$, 都有

$$\Pr[|f(X_1, \dots, X_m) - \mathbb{E}[f(X_1, \dots, X_m)]| \geq t] \leq 2e^{-2t^2/(mL^2)}.$$

To apply McDiarmid's inequality to our situation, define $f(r_1, \dots, r_m) := q$ to be the number of overflow balls. Observe that f is n^δ -Lipschitz, since each r_i can determine the outcome of at most n^δ different balls. Since $\mathbb{E}[q] = \frac{1}{\text{poly } n}$, it follows by McDiarmid's inequality that

为了把 McDiarmid 不等式应用到当前问题中, 我们定义 $f(r_1, \dots, r_m) := q$, 即 overflow balls 的总数。注意到 f 是 n^δ -Lipschitz 的, 因为每个 r_i 最多只能影响 n^δ 个不同 balls 的命运。再结合 $\mathbb{E}[q] = 1/\text{poly } n$, 由 McDiarmid 不等式可得

$$\begin{aligned} \Pr[f(r_1, \dots, r_m) \geq n^{1-\delta}] &\leq e^{-\Omega(n^{2-2\delta}/(mn^{2\delta}))} \\ &= e^{-\Omega(n^{2-2\delta}/n^{1+2\delta})} \\ &= e^{-\Omega(n^{1-4\delta})}. \end{aligned}$$

For any $0 < \epsilon \leq 1$, we can set $\delta = \epsilon/5$ so that

$$\begin{aligned} \Pr[q \geq n^{1-\delta}] &\leq e^{-\Omega(n^{1-4\delta})} \\ &\leq O\left(n^{-n^{1-\epsilon}}\right). \end{aligned}$$

对任意 $0 < \epsilon \leq 1$, 只需令 $\delta = \epsilon/5$, 就能进一步得到

$$\begin{aligned} \Pr[q \geq n^{1-\delta}] &\leq e^{-\Omega(n^{1-4\delta})} \\ &\leq O\left(n^{-n^{1-\epsilon}}\right). \end{aligned}$$

This establishes (1). If we implement deletions as in Section 2, then we arrive at the following theorem.

这就证明了式 (1)。如果再按第 2 节所述方式处理 deletions, 就得到下面的定理。

Theorem 2. *The $n^{\epsilon/5}$ -ary amplified rotated radix trie is a randomized linear-space dictionary that can store up to $n \Theta(\log n)$ -bit keys/values at a time, and that supports each operation in constant time with probability $1 - O(1/n^{n^{1-\epsilon}})$.*

也就是说, $n^{\epsilon/5}$ 叉的 amplified rotated radix trie 是一个 randomized、linear-space 的 dictionary。它可以同时存储最多 n 个 $\Theta(\log n)$ 比特的 key/value 对, 并且每次操作都以 $1 - O(1/n^{n^{1-\epsilon}})$ 的概率在常数时间内完成。

We remark that there is a strong sense in which the amplified rotated radix trie is nearly optimal. In particular, for any constant $\epsilon > 0$, if there were to exist a randomized linear-space dictionary with failure probability of $1/n^{\epsilon n}$, that would imply the existence of a deterministic (though non-explicit) linear-space constant-time dictionary.

我们还指出, 在一个很强的意义下, amplified rotated radix trie 已经接近最优。具体来说, 对任意常数 $\epsilon > 0$, 如果存在某个 randomized linear-space dictionary, 其 failure probability 达到 $1/n^{\epsilon n}$, 那么这将推出一个 deterministic (虽然未必 explicit) 的 linear-space constant-time dictionary 的存在。

Lemma 3. *Let $\epsilon > 0$ be any positive constant and assume a machine word of size $w = \Theta(\log n)$ bits. Suppose there exists randomized linear-space dictionary that stores up to $n \Theta(\log n)$ -bit keys/values at a time and has failure probability $1/n^{\epsilon n}$. Then there also exists a deterministic (not-necessarily explicit) dictionary with the same guarantees.*

该引理的含义是: 若在 word size 为 $\Theta(\log n)$ 比特的标准设定下, 存在一个 randomized linear-space dictionary, 能同时存储至多 n 个 $\Theta(\log n)$ 比特的 key/value 对, 并具有 $1/n^{\epsilon n}$ 的 failure probability, 那么也必然存在一个 deterministic (不一定 explicit) dictionary, 具备相同量级的保证。

Proof. To distinguish the randomized dictionary from the deterministic dictionary that we are constructing, we will refer to the former as a hash table and the latter as a dictionary.

As noted in Section 2, by rebuilding our dictionary once every $O(n)$ operations, we can assume without loss of generality that the lifespan of the dictionary is at most $O(n)$ operations. We will implement the dictionary using a hash table with capacity $n' = cn$ for some large positive constant c to be determined later. This means that the hash table has failure probability

$$1/n^{\epsilon n'} = 1/n^{\epsilon cn}.$$

Each operation takes place on a $\Theta(\log n)$ -bit key/value pair, so there are at most $n^{O(1)}$ options for what a given operation could be. The total number of $O(n)$ -long operation sequences is therefore at most $n^{O(n)}$. Since our hash table has failure probability $1/n^{\epsilon cn}$, its total failure probability on any given sequence of $O(n)$ operations is at most $O(n)/n^{\epsilon cn} \leq 1/n^{\epsilon cn/2}$. The probability that there exists *any* sequence of operations on which our hash table fails to be constant-time is therefore at most

$$\frac{n^{O(1)}}{n^{\epsilon cn/2}},$$

which if c is taken to be a sufficiently large constant, is at most $1/2$. Thus there exists some choice of random bits for which our hash table is constant-time on *every* sequence of operations. By hard-coding in this choice of random bits, we arrive at a deterministic constant-time dictionary.

Note that, since the hash table spends total time $O(n)$ on the $O(n)$ operations, the number of random bits that it can use is at most $O(nw) = O(n \log n)$ bits—thus the deterministic dictionary can hard-code the random bits in linear space. $\square$

证明思路是一个典型的计数加 union bound。为了区分对象，我们把原来的 randomized dictionary 称为 hash table，而把要构造的 deterministic 版本称为 dictionary。

正如第 2 节所述，通过每隔 $O(n)$ 次操作重建一次字典，我们可以不失一般性地假设该 dictionary 的生命周期最多只有 $O(n)$ 次操作。接着，我们使用一个容量为 $n' = cn$ 的 hash table 来实现目标 dictionary，其中 c 是稍后选取的一个充分大的正常数。这时 hash table 的 failure probability 就是

$$1/n^{\epsilon n'} = 1/n^{\epsilon cn}.$$

由于每次操作都作用于一个 $\Theta(\log n)$ 比特的 key/value 对，因此单次操作的可能性只有 $n^{O(1)}$ 种，从而长度为 $O(n)$ 的操作序列总数至多为 $n^{O(n)}$ 。而对任意固定的一个 $O(n)$ 长操作序列，这个 hash table 的总失败概率至多为 $O(n)/n^{\epsilon cn} \leq 1/n^{\epsilon cn/2}$ 。于是，存在某个操作序列使 hash table 退化出非常数时间的概率，至多为

$$\frac{n^{O(1)}}{n^{\epsilon cn/2}}.$$

只要 c 取得足够大，这个值就会小于等于 $1/2$ 。这就意味着，必然存在某组 random bits，使得这个 hash table 在所有操作序列上都保持常数时间。把这组 random bits 直接硬编码进去，就得到了一个 deterministic constant-time dictionary。

最后注意，由于这个 hash table 在 $O(n)$ 次操作上总共只花 $O(n)$ 时间，因此它最多只能读取 $O(nw) = O(n \log n)$ 个 random bits，所以 deterministic dictionary 也只需在线性空间内把这些 random bits 硬编码进去即可。

Although one typically assumes a machine-word size of $\Theta(\log n)$ bits, it is also an interesting question what the strongest achievable probabilistic guarantees are in the setting where machine words (as well as keys/values) are of some size $w = \omega(\log n)$ bits. On one hand, the larger key size makes it so that Lemma 3 no longer applies, so in principle, one might be able to achieve a failure probability of $1/n^{\omega(n)}$. On the other hand, from an upper-bound perspective, it is not even known how to achieve a *sub-polynomial failure probability* in this setting [3, 16, 17, 29]. Here, the main obstacle appears to be unavoidably about hash functions: can one construct a family of hash functions from $[2^w]$ to $[\text{poly}(n)]$ such that for any given n -element set $S \subseteq [2^w]$, we have that $\max_{x \in S} |\{y \in S \mid h(x) = h(y)\}| \leq \text{polylog } n$ with probability $1/n^{\omega(1)}$? If such a family were to exist, then it could be directly combined with Theorem 2 to construct a dictionary that achieves sub-polynomial failure probability for any key-size w . We conjecture that no such family of hash functions exists, and moreover, that a sub-polynomial failure probability is not possible for word sizes $w = \omega(\log n)$ bits.

虽然通常人们默认 machine-word size 为 $\Theta(\log n)$ 比特，但另一个很有意思的问题是：当 machine words（以及 keys/values）长度达到 $w = \omega(\log n)$ 时，究竟还能获得多强的概率保证。一方面，由于 key 更长，Lemma 3 已经不再适用，因此理论上 failure probability 也许可以达到 $1/n^{\omega(n)}$ 。另一方面，从上界构造的角度看，人们目前甚至还不知道在这个 setting 下如何实现次多项式的 failure probability [3, 16, 17, 29]。这里的核心障碍看起来仍然不可避免地落在 hash functions 上：是否存在某个从 $[2^w]$ 到 $[\text{poly}(n)]$ 的 hash family，使得对任意固定的 n 元集合 $S \subseteq [2^w]$ ，都有 $\max_{x \in S} |\{y \in S \mid h(x) = h(y)\}| \leq \text{polylog } n$ ，并且该事件发生的概率为 $1/n^{\omega(1)}$ ？如果这样的 hash

family 存在，那么就可以直接把它与 Theorem 2 结合，从而对任意 key-size w 构造出具有次多项式 failure probability 的 dictionary。我们猜测这样的 hash family 并不存在，而且更进一步地猜测：当 word size 满足 $w = \omega(\log n)$ 时，次多项式 failure probability 本身就是不可能的。

5 The Budget Rotated Trie

In this section, we present a dictionary that uses only $O(\log n \log \log n)$ random bits, while guaranteeing that each operation takes constant time with probability $1 - 1/\text{poly}(n)$ (i.e., with high probability in n). We will refer to the data structure as the ***budget rotated trie***. In Appendix A, we further extend the budget rotated trie to support keys that are $\omega(\log n)$ bits, while still using only $O(\log n \log \log n)$ bits of randomness.

本节中，我们给出一种只使用 $O(\log n \log \log n)$ 个 random bits 的 dictionary，同时仍保证每次操作都以 $1 - 1/\text{poly}(n)$ 的概率在常数时间内完成，也就是对 n 而言以 high probability 成立。我们把该数据结构称为 ***budget rotated trie***。在附录 A 中，我们还会把它推广到 key 长度为 $\omega(\log n)$ 比特的情形，同时仍然只使用 $O(\log n \log \log n)$ 个随机位。

We remark that the guarantee achieved by the budget rotated trie is not novel—in fact, a previous approach by Dietzfelbinger, Gil, Matias, and Pippenger [10] can be used to achieve $O(\log n)$ random bits for the setting of $\Theta(\log n)$ -bit keys that we are considering. Nonetheless, we believe that the construction for the budget rotated tries is interesting in its own right, both because of its relationship to the amplified rotated trie, and also because of the surprising way in which it is able to make use of gradually-increasing-independence hash functions. Additionally, the specific structure of the budget rotated trie will prove useful in our quest for succinctness in Section 6.

需要说明的是，budget rotated trie 实现的这种 guarantee 本身并不是全新的。事实上，在我们讨论的 $\Theta(\log n)$ 比特 keys 的设定下，Dietzfelbinger、Gil、Matias 和 Pippenger [10] 的更早方案也能做到 $O(\log n)$ 个 random bits。尽管如此，我们仍然认为 budget rotated trie 的构造本身很有意思：一方面，它与 amplified rotated trie 有紧密联系；另一方面，它竟然能够以一种相当出人意料的方式利用 gradually-increasing-independence hash functions。此外，budget rotated trie 的这种特殊结构还会在第 6 节追求 succinctness 时发挥关键作用。

Our starting place is again the rotated trie, and as in Section 4, we will take the fanout of the trie to be n^δ for some constant δ ; in fact, it will suffice to simply use $\delta = 1/4$.

我们的出发点仍然是 rotated trie，并且和第 4 节一样，把 trie 的 fanout 设为 n^δ ，其中 δ 是某个常数。事实上，取 $\delta = 1/4$ 就已经足够。

Reducing the number of random bits to $O(n/\text{polylog } n)$. To transform the n^δ -ary rotated trie into a budget rotated trie, our first modification will be to reduce the number of random bits from $O(n \log n)$ to $O(n/\text{polylog } n)$. Of course, this may not seem like much progress, but we shall see later that the distinction is important.

为了把 n^δ 叉 rotated trie 变成 budget rotated trie，我们的第一步修改是把 random bits 的数量从 $O(n \log n)$ 降到 $O(n/\text{polylog } n)$ 。这看起来似乎进步不大，但后面会看到，这种区别实际上非常关键。

Recall that, in a rotated trie, each ball b (i.e., each non-null entry in an internal node) contains a pointer to either a leaf (i.e., an actual key/value pair) or another internal node (i.e., a child). We now add a third option: if the ball should be pointing at another internal node x , but if the subtree rooted at x contains fewer than $\ell = \text{polylog } n$ total keys, then we store that subtree as a dynamic fusion node z . If the size of the subtree rooted at x subsequently surpasses ℓ , then we create an actual internal node for x —in this case, any elements stored in the fusion node z remain in z , and the ball b now stores two pointers, one to x and one to z . In other words, there are now three possible states for a ball: it can contain a pointer to a leaf; it can contain a pointer to a dynamic fusion node; or it can contain two pointers, one to a dynamic fusion node and one to another internal node of the trie.

回忆一下，在 rotated trie 中，每个 ball b （也就是 internal node 中的每个非空条目）原本都保存着一个指针，这个指针要么指向一个 leaf（即真实的 key/value 对），要么指向另一个 internal node（即 child）。现在我们加入第三种可能：如果某个 ball 原本应该指向另一个 internal node x ，但 x 为根的子树中总共还不到 $\ell = \text{polylog } n$ 个 keys，那么我们就直接把这棵子树压成一个 dynamic fusion node z 。如果之后该子树规模超过了 ℓ ，我们再真正创建 internal node x ；这时，原先存放在 fusion node z 中的元素仍然保留在 z 中，而 ball b 则开始同时保存两个指针，一个指向 x ，另一个指向 z 。换句话说，此时一个 ball 会有三种状态：指向 leaf；指向 dynamic fusion node；或者同时指向一个 dynamic fusion node 和一个 trie 的 internal node。

The point of this modification is that we only create an internal node x if the subtree rooted at x contains at least $\ell = \text{polylog } n$ elements. Importantly, this means that the total number of internal nodes m is at most $O(n/\ell) = n/\text{polylog } n$. The number of random bits needed for the rotations $r_1, r_2, \dots, r_m$ is therefore also $n/\text{polylog } n$.

这一修改的目的在于：只有当以 x 为根的子树中至少已有 $\ell = \text{polylog } n$ 个元素时，我们才真正创建 internal node x 。这非常重要，因为它意味着 internal nodes 的总数 m 最多只有 $O(n/\ell) = n/\text{polylog } n$ 。因此，用于 rotations $r_1, r_2, \dots, r_m$ 的 random bits 总数也会同步下降到 $n/\text{polylog } n$ 。

Changing the balls-to-bins mapping. Our next modification is to change how we map the balls to bins. Recall that each ball b is specified by a pair (s, c) , where $s \in [m]$ is the source node of the ball and $c \in [n^\delta]$ is the child index. In the standard rotated trie, we map balls to bins using the function

$$\phi(s, c) = (c + r_s) \bmod n.$$

下一步修改是改变 balls 到 bins 的映射方式。回忆一下，每个 ball b 由一对 (s, c) 决定，其中 $s \in [m]$ 是该 ball 的 source node， $c \in [n^\delta]$ 是 child index。在标准 rotated trie 中，我们使用函数

$$\phi(s, c) = (c + r_s) \bmod n.$$

把 balls 映射到 bins。

We will now instead map balls to bins using the function

$$\psi(s, c) = (c + a_s \bmod n^\delta) \cdot n^{1-\delta} + b_s,$$

where a_s is selected at random from $[n^\delta]$ and b_s is selected at random from $[n^{1-\delta}]$.

现在我们改用下面的函数来做 balls-to-bins mapping:

$$\psi(s, c) = (c + a_s \pmod{n^\delta}) \cdot n^{1-\delta} + b_s,$$

其中 a_s 从 $[n^\delta]$ 中随机选取, b_s 从 $[n^{1-\delta}]$ 中随机选取。

When can think about ψ as follows. We break the bins into groups $G_1, \dots, G_{n^\delta}$ of size $n^{1-\delta}$, and we use the random value $a_s \in [n^\delta]$ to assign the ball to a random group. Once the ball is assigned to a group G_i , it is then assigned to the b_s -th bin in that group. Importantly, the assignments are designed so that each source node s assigns *at most one* of its balls to any given group G_i . There will never be two balls b_1, b_2 in group G_i that both obtain their assignments b_s from the same source node.

可以这样理解 ψ 。我们先把所有 bins 划分成 n^δ 个 groups, 记为 $G_1, \dots, G_{n^\delta}$, 每个 group 的大小都是 $n^{1-\delta}$ 。然后利用随机值 $a_s \in [n^\delta]$ 把某个 ball 分配到一个随机 group 中; 一旦进入某个 group G_i , 它再被放到该 group 中第 b_s 个 bin 上。最关键的是, 这种设计保证了: 同一个 source node s 的 balls 在任何固定 group G_i 中至多只会出现一个。也就是说, 不会有两个 balls b_1, b_2 同时位于同一 group G_i , 并且它们的最终位置都由同一个 source node 的 b_s 决定。

Since the number m of internal nodes may be as large as $n/\text{polylog } n$, we cannot afford to generate $a_1, a_2, \dots, a_m \in [n^\delta]$ and $b_1, b_2, \dots, b_m \in [n^{1-\delta}]$ truly at random. Fortunately, as we shall now see, the roles of the a_i s and b_i s have been carefully designed so that both sequences can be generated using a small number of “seed” random bits.

由于 internal nodes 的数量 m 可能大到 $n/\text{polylog } n$, 我们无法承受把 $a_1, a_2, \dots, a_m \in [n^\delta]$ 以及 $b_1, b_2, \dots, b_m \in [n^{1-\delta}]$ 全都真正地独立均匀随机生成。幸运的是, 正如接下来会看到的, 这里对 a_i 和 b_i 的角色划分被设计得非常巧妙, 使得这两组序列都可以由很少的 “seed” random bits 生成。

Generating the a_i s with $O(1)$ -independent hash functions. Let k be a sufficiently large positive constant, and select a random hash function $g : [n] \rightarrow [n^\delta]$ from a family of k -independent hash functions. Since $k = O(1)$, the function g can be specified using $O(\log n)$ random bits, and can be evaluated in time $O(1)$. We compute the a_i s by

$$a_i := g(i).$$

令 k 为一个足够大的正常数, 并从某个 k -independent hash function family 中随机选取一个函数 $g : [n] \rightarrow [n^\delta]$ 。由于 $k = O(1)$, 该函数 g 只需要 $O(\log n)$ 个 random bits 就能描述, 并且可以在 $O(1)$ 时间内求值。然后我们令

$$a_i := g(i).$$

To analyze the number of balls in each group G_i , we use a well-known tail bound for k -independent random variables (see, e.g., [2] or [12]).

为了分析每个 group G_i 中的 balls 数量, 我们要使用一个关于 k -independent 随机变量的经典尾界 (例如见 [2] 或 [12])。

Lemma 4 (Lemma 2.2 of [2]). *Let $k \geq 4$ be an even integer. Suppose $X_1, \dots, X_m$ are k -wise independent 0-1 random variables. Let $X = \sum_i X_i$. Then, for any $t \geq 0$,*

$$\Pr[|X - \mathbb{E}[X]| \geq t] \leq 2 \left(\frac{nk}{t^2} \right)^{k/2}.$$

这个引理说明：即使随机变量只满足 k -wise independence，也仍然可以对它们的和得到一个足够强的 tail bound，用来近似替代 Chernoff 风格的集中界。

Define X_j to be the event that source-node j sends a ball to group G_i . The X_j s are k -independent, so we have by Lemma 4 that

$$\Pr[|G_i| - \mathbb{E}[|G_i|] \geq n^{0.75}] \leq 2 \left(\frac{kn}{n^{1.5}} \right)^{k/2} \leq n^{-\Omega(k)} = 1/\text{poly}(n).$$

Since $\delta = 0.25$, it follows that

$$\Pr[|G_i| - \mathbb{E}[|G_i|] \geq n^{1-\delta}] \leq 1/\text{poly}(n).$$

Since each of the $O(n)$ balls is equally likely to be in any group, we have that $\mathbb{E}[|G_i|] = O(n^{1-\delta})$. Thus

$$\Pr[|G_i| \leq O(n^{1-\delta})] \geq 1 - 1/\text{poly}(n).$$

That is, each group G_i contains at most $O(n^{1-\delta})$ balls with high probability in n .

令 X_j 表示 source-node j 是否向 group G_i 发送了一个 ball 的事件。由于这些 X_j 是 k -independent 的，由 Lemma 4 可得

$$\Pr[|G_i| - \mathbb{E}[|G_i|] \geq n^{0.75}] \leq 2 \left(\frac{kn}{n^{1.5}} \right)^{k/2} \leq n^{-\Omega(k)} = 1/\text{poly}(n).$$

又因为这里我们取 $\delta = 0.25$ ，于是进一步有

$$\Pr[|G_i| - \mathbb{E}[|G_i|] \geq n^{1-\delta}] \leq 1/\text{poly}(n).$$

另一方面，由于总共 $O(n)$ 个 balls 对所有 groups 来说是均匀分布的，所以 $\mathbb{E}[|G_i|] = O(n^{1-\delta})$ 。因此

$$\Pr[|G_i| \leq O(n^{1-\delta})] \geq 1 - 1/\text{poly}(n).$$

也就是说，以 high probability，每个 group G_i 中的 balls 数量都不会超过 $O(n^{1-\delta})$ 。

Generating the b_i s with increasing-independence hash functions. To generate the b_i s without using a large number of random bits, we make use of a more sophisticated family of hash functions. Call a family $\mathcal{H}(t)$ of hash functions $h : [\text{poly}(t)] \rightarrow [t]$ **load-balancing** if it can be used

to map t balls to t bins with maximum load $\text{polylog } t$; that is, for any fixed set $S \subseteq \text{poly}(t)$ of size t , and for any fixed $i \in [t]$, if we select a random $h \in \mathcal{H}$, then

$$|\{s \in S \mid h(s) = i\}| \leq \text{polylog } t$$

with probability $1 - 1/\text{poly}(t)$.

为了在不消耗太多 random bits 的前提下生成 b_i , 我们要使用一种更精巧的 hash function family。若某个 family $\mathcal{H}(t)$ 中的函数 $h : [\text{poly}(t)] \rightarrow [t]$ 能够把 t 个 balls 映射到 t 个 bins, 并保证最大负载为 $\text{polylog } t$, 那么我们就称它是 **load-balancing** 的。更具体地说, 对任意大小为 t 的固定集合 $S \subseteq \text{poly}(t)$ 以及任意固定的 bin $i \in [t]$, 若随机选取 $h \in \mathcal{H}$ 后都有

$$|\{s \in S \mid h(s) = i\}| \leq \text{polylog } t$$

并且这一事件的概率至少为 $1 - 1/\text{poly}(t)$, 那么这个 family 就满足我们的要求。

Celis, Reingold, Segev, and Wieder [7] showed how to construct a load-balancing family $\mathcal{H}(t)$ of hash functions such that each $h \in \mathcal{H}$ can be described with $O(\log t \log \log t)$ random bits and can be evaluated in time $O(\log t \log \log t)$. The family $\mathcal{H}$ is referred to as having “gradually-increasing-independence” because each $h \in \mathcal{H}$ is actually the composition of $\Theta(\log \log t)$ hash functions $h_1, \dots, h_{\Theta(\log \log t)}$ with different levels of independence: each h_i determines $\Theta((3/4)^i \log t)$ bits of h , and each h_i is $(1/\text{poly } t)$ -close to being $\Theta((4/3)^i)$ -independent.

Celis, Reingold, Segev 和 Wieder [7] 给出了一种 load-balancing hash family $\mathcal{H}(t)$, 其中每个 $h \in \mathcal{H}$ 都只需要 $O(\log t \log \log t)$ 个 random bits 来描述, 并且求值时间为 $O(\log t \log \log t)$ 。所以说这个 family 具有 “gradually-increasing-independence”, 是因为每个 $h \in \mathcal{H}$ 实际上都是 $\Theta(\log \log t)$ 个 hash functions $h_1, \dots, h_{\Theta(\log \log t)}$ 的复合, 而且这些函数的独立性水平逐步上升: 每个 h_i 会决定 h 的 $\Theta((3/4)^i \log t)$ 个比特, 并且各自都是 “距离 $\Theta((4/3)^i)$ -independent 只差 $1/\text{poly } t$ ” 的。

The family $\mathcal{H}$ comes with a tradeoff. It is able to achieve a maximum load of $\text{polylog } t$ (in fact, it even achieves maximum load $O(\log t / \log \log t)$) using on $O(\log t \log \log t)$ bits, but it requires super-constant time to evaluate. Subsequent work [23] has improved the evaluation time from $O(\log t \log \log t)$ to $O((\log \log t)^2)$. It seems unlikely that the evaluation time can be improved to $O(1)$, however, since $\Omega(\log \log t)$ time is needed just to read the random bits used to evaluate the hash function.

这个 family $\mathcal{H}$ 带来了一个典型 tradeoff。它能够只用 $O(\log t \log \log t)$ 个 bits, 就把最大负载控制在 $\text{polylog } t$ (事实上甚至可以做到 $O(\log t / \log \log t)$); 但代价是它的求值时间是超常数的。之后的工作 [23] 又把求值时间从 $O(\log t \log \log t)$ 改进到 $O((\log \log t)^2)$ 。不过, 求值时间要进一步降到 $O(1)$ 看起来并不现实, 因为仅仅读取参与求值的 random bits, 本身就至少需要 $\Omega(\log \log t)$ 时间。

The super-constant evaluation time makes it so that hash functions with gradually-increasing independence are not suitable for direct use in constant-time hash tables [32]. We get around this problem by using h not as a hash function but as a pseudo-random number generator. Specifically, we select a random $h : [m] \rightarrow [n^{1-\delta}]$ from $\mathcal{H}(n^{1-\delta})$, and we use h to initialize the b_i s as

$$b_i := h(i).$$

由于求值时间是超常数的, 这种 gradually-increasing-independence hash function 并不适合直接用在 constant-time hash table 的热路径中 [32]。我们的绕开方式是: 不把 h 当作真正的 hash function 去频繁求值, 而是把它当作 pseudo-random number generator。具体地说, 我们从 $\mathcal{H}(n^{1-\delta})$ 中随机选取一个函数 $h : [m] \rightarrow [n^{1-\delta}]$, 并用它来初始化

$$b_i := h(i).$$

Since h takes time $O((\log \log n)^2)$ to evaluate, each b_i now takes time $O((\log \log n)^2)$ to initialize. Recall, however, that we only create a new internal node x in our rotated trie once there are more than $\ell = \text{polylog } n$ records that want to reside in that node's subtree; the first $\ell = \text{polylog } n$ insertions that wish to use x are instead placed into a dynamic fusion node that acts as a proxy for x . As a result, we can afford to spend up to ℓ time *initializing* the node x , and we can spread that time across the ℓ insertions that trigger x 's initialization. Since $\ell = \omega((\log \log n)^2)$, we can initialize $b_i = h(i)$ without any problem.

由于 h 的求值时间是 $O((\log \log n)^2)$, 因此每个 b_i 的初始化时间也变成了 $O((\log \log n)^2)$ 。不过请回忆: 在 rotated trie 中, 只有当某个子树中的记录数超过 $\ell = \text{polylog } n$ 时, 我们才真正创建新的 internal node x ; 在那之前, 最初想要进入这棵子树的前 ℓ 次插入都会先被放进一个充当 proxy 的 dynamic fusion node 里。因此, 我们完全可以把最多 ℓ 的初始化代价分摊到这 ℓ 次触发创建的插入操作上。由于 $\ell = \omega((\log \log n)^2)$, 初始化 $b_i = h(i)$ 并不会带来问题。

Analyzing the maximum load. Recall that, with probability $1 - 1/\text{poly}(n)$, each group G_i contains at most $O(n^{1-\delta})$ balls. Furthermore, each of the balls have different source nodes than one another. If a ball has source-node s , then it is placed in the b_s -th bin of G_i .

回忆一下, 我们已经证明: 以 $1 - 1/\text{poly}(n)$ 的概率, 每个 group G_i 中的 balls 数量至多为 $O(n^{1-\delta})$ 。并且同一 group 中这些 balls 的 source nodes 两两不同。若某个 ball 的 source-node 为 s , 那么它会被放入 group G_i 的第 b_s 个 bin 中。

Let $S_i \subseteq [m]$ be the set of source nodes that assign balls to G_i . Then for each $r \in [n^{1-\delta}]$, the number $g_{i,r}$ of balls in the r -th bin of G_i is given by

$$g_{i,r} = |\{s \in S_i \mid h(s) = r\}|.$$

Since $h : \text{poly}(n) \rightarrow [n^{1-\delta}]$ is from a load-balancing family of hash functions, we are guaranteed to have

$$g_{i,r} \leq \text{polylog } n^{1-\delta} \leq \text{polylog } n$$

with high probability in n .

设 $S_i \subseteq [m]$ 是所有把 balls 发送到 group G_i 的 source nodes 集合。那么对任意 $r \in [n^{1-\delta}]$, group G_i 中第 r 个 bin 的 balls 数量满足

$$g_{i,r} = |\{s \in S_i \mid h(s) = r\}|.$$

由于 $h : \text{poly}(n) \rightarrow [n^{1-\delta}]$ 来自一个 load-balancing hash family, 因此我们能够保证

$$g_{i,r} \leq \text{polylog } n^{1-\delta} \leq \text{polylog } n$$

并且该结论以关于 n 的 high probability 成立。

Putting the pieces together. The fact that each bin contains at most $\text{polylog } n$ balls (with high probability) means that, as in the standard rotated trie, each bin can be implemented with a dynamic fusion node. Operations on our dictionary therefore take time $O(1)$ with high probability in n . If we implement deletions as in Section 2, then we arrive at the following theorem.

既然每个 bin 都以 high probability 至多只包含 $\text{polylog } n$ 个 balls, 那么和标准 rotated trie 一样, 我们就可以继续用 dynamic fusion node 来实现每个 bin。于是, 整个 dictionary 的各项操作都能以关于 n 的 high probability 在 $O(1)$ 时间内完成。若再按照第 2 节所述方式处理 deletions, 就得到下面的定理。

Theorem 5. *The budget rotated trie is a randomized linear-space dictionary that can store up to $n \Theta(\log n)$ -bit keys/values at a time, that uses $O(\log n \log \log n)$ random bits, and that supports each operation in constant time with probability $1 - 1/\text{poly}(n)$.*

也就是说, budget rotated trie 是一个 randomized、linear-space 的 dictionary。它能够同时存储至多 n 个 $\Theta(\log n)$ 比特的 key/value 对, 只使用 $O(\log n \log \log n)$ 个 random bits, 并且每次操作都以 $1 - 1/\text{poly}(n)$ 的概率在常数时间内完成。

We conclude the section by observing that there is a strong sense in which the guarantee achieved by the budget rotated trie is optimal. In particular, if there were to exist a hash table failure probability $1/n^c$ but that used fewer than $c \log n$ random bits, then there would also necessarily exist a deterministic linear-space constant-time dictionary.

本节最后指出, budget rotated trie 所达到的 guarantee 在一个很强的意义下也是最优的。具体来说, 如果存在某个 hash table, 它的 failure probability 达到 $1/n^c$, 但所使用的 random bits 却少于 $c \log n$, 那么这同样会推出 deterministic linear-space constant-time dictionary 的存在。

Lemma 6. *Suppose there exists a randomized linear-space dictionary that can store up to $n \Theta(\log n)$ -bit keys/values at a time, that uses $c \log n$ random bits, but that has a failure probability smaller than $1/n^c$. Then there exists a deterministic dictionary with the same guarantees.*

这个引理的意思是: 如果某个 randomized linear-space dictionary 同时满足 “只用 $c \log n$ 个 random bits” 和 “failure probability 严格小于 $1/n^c$ ”, 那么就一定能推出一个具有相同保证的 deterministic dictionary。

Proof. To distinguish the randomized dictionary from the deterministic dictionary that we are constructing, we will refer to the former as a hash table. Let R denote the $c \log n$ random bits used by the hash table. Define $\mathcal{D}$ to be the deterministic dictionary obtained by setting $R = 0$. Suppose for contradiction that $\mathcal{D}$ is not constant time. Then there exists some sequence of operations such that the final operation on $\mathcal{D}$ takes super-constant time. This means that, with probability at least $1/n^c$, that same operation would have taken super-constant time in our hash table. But the hash table has failure probability smaller than $1/n^c$, a contradiction. $\square$

证明很短。为了区分对象，我们仍把 randomized 版本称为 hash table。设该 hash table 使用的 random bits 为 R ，总数为 $c \log n$ 。如果把这些 random bits 全部固定成某个具体取值，就得到一个 deterministic dictionary。现在反设每个这样的 deterministic dictionary 都不是 constant time，那么至少存在某个操作序列，使得最终一步在其中一个固定取值下退化为超常数时间。可是一旦如此，对均匀随机选取的 R 来说，这同一步骤以至少 $1/n^c$ 的概率也会退化成超常数时间，这就与原 hash table 的 failure probability 严格小于 $1/n^c$ 矛盾。

6 Achieving Succinctness

In this section, we turn our attention to space efficiency. Throughout the section, we will use $c_1 \log n$ to denote the size in bits of each key, we will use $c_2 \log n$ to denote the size in bits of each value, and will assume that $c = c_1 + c_2$ is a positive constant larger than 1. Here, unlike in previous sections, we use n to denote the *current size*, at any given moment, and we allow n to change dynamically over time, so long as the key/value length remains $\Theta(\log n)$ bits at all times after initialization—this means that the constants c_1, c_2, c also change dynamically.⁷

本节中，我们把注意力转向空间效率。贯穿本节，记每个 key 的长度为 $c_1 \log n$ 比特，每个 value 的长度为 $c_2 \log n$ 比特，并假设 $c = c_1 + c_2 > 1$ 是一个正常数。与前文不同，这里 n 表示任意时刻的当前大小，因此允许它随时间动态变化；只要求在初始化之后，key/value 的长度始终保持为 $\Theta(\log n)$ 比特即可。这也意味着常数 c_1, c_2, c 会随时间一起变化。⁸

The dictionaries that we have described in previous sections are already optimal up to constant factors, using a total of $\Theta(n \log n)$ bits to store n key/value pairs. We shall now strive to achieve optimal space consumption up to low-order terms, that is, to use a total of

$$(1 + o(1)) \log \binom{2^{c \log n}}{n} = cn \log n - n \log n + o(n \log n) \quad (2)$$

bits. Such a dictionary is referred to as **succinct** [31].

前几节给出的 dictionaries 在常数因子意义下已经是最优的，它们用总共 $\Theta(n \log n)$ 比特来存储 n 个 key/value 对。现在我们的目标是把空间进一步压缩到只差低阶项，也就是总空间达到

$$(1 + o(1)) \log \binom{2^{c \log n}}{n} = cn \log n - n \log n + o(n \log n) \quad (3)$$

比特。达到这一空间级别的 dictionary 就被称为 **succinct** [31]。

In fact, we will prove a much more general result: that any constant-time dictionary can be *transformed* into a succinct constant-time dictionary, while (nearly) preserving the random-bit usage and failure probability of the original dictionary.

⁷Note that keys trivially must have length at least $\log n$ bits, so the key/value size is necessarily $\Omega(\log n)$. If we assume that values are asymptotically no larger than keys, then one can enforce the bound of keys/values having length $O(\log n)$ by treating the dictionary as containing an extra $2^{\epsilon w}$ dummy elements, where w is the key/value length and ϵ is a small positive constant.

⁸注意，key 的长度平凡地至少要有 $\log n$ 比特，因此 key/value 总长度必然是 $\Omega(\log n)$ 。如果进一步假设 value 的渐近大小不超过 key，那么还可以通过在 dictionary 中附加 $2^{\epsilon w}$ 个 dummy elements 的方式，把 key/value 长度上界强制控制在 $O(\log n)$ ，其中 w 为 key/value 长度， ϵ 为一个小正常数。

事实上，我们将证明一个更一般的结果：任意 constant-time dictionary 都可以被转换成一个 succinct constant-time dictionary，并且原结构使用的 random bits 数量与 failure probability 都几乎可以被保留下来。

Define an $(r(n), p(n), s(n))$ -**dictionary** to be any constant-time dictionary that uses $O(r(n))$ random bits, achieves failure probability $O(p(n))$ per operation, and uses space $cn \log n - n \log n + O(s(n))$ bits. Since we are interested in dictionaries that automatically resize as the number of keys change, one should think of $r(n)$, $p(n)$, and $s(n)$ as functions rather than fixed values. Note that, in the context of space-efficient time-efficient dictionaries, a failure event could be in terms of either time (an operation takes $\omega(1)$ time) or space (the dictionary fails to fit in $cn \log n - n \log n + O(s(n))$ bits)—and a failure probability of $p(n)$ means that, at any given moment, the probability of a failure occurring should be at most $O(p(n))$.

我们把一个 constant-time dictionary 称为 $(r(n), p(n), s(n))$ -**dictionary**，如果它满足：使用 $O(r(n))$ 个 random bits；每次操作的 failure probability 为 $O(p(n))$ ；并且占用空间为 $cn \log n - n \log n + O(s(n))$ 比特。由于我们关心的是那种随着 key 数量变化而自动调整大小的 dictionaries，因此这里的 $r(n)$ 、 $p(n)$ 和 $s(n)$ 都应该理解为函数而不是固定常数。还要注意，在“时间高效且空间高效”的 dictionary 语境下，failure 既可能发生在时间上（某次操作耗时 $\omega(1)$ ），也可能发生在空间上（字典无法装进 $cn \log n - n \log n + O(s(n))$ 比特空间）。因此，failure probability 为 $p(n)$ 的意思是：在任意给定时刻，发生任意一种 failure 的概率都至多为 $O(p(n))$ 。

The main result of the section can be stated as follows.

本节的主结果可以表述如下。

Theorem 7. *Let ϵ be a small positive constant. Suppose that $r(n)$ and $p(n)$ are nondecreasing functions satisfying $r(n) \leq O(n)$ and $\exp(-n^{1-\epsilon}) \leq p(n) \leq 1/\text{polylog}(n)$. Given an $(r(n), p(n), n \log n)$ -dictionary, one can construct a $(r'(n), p'(n), s'(n))$ -dictionary with*

$$r'(n) = r(n) + (\log p(n)^{-1}) \cdot (\log \log n)^3,$$

$$p'(n) = p(n)/\log \log n,$$

and

$$s'(n) = \frac{n \log n}{\log \log n}.$$

这个定理说明：只要起点是一个空间冗余为 $n \log n$ 的 dictionary，并且其 random bits 用量和 failure probability 落在相应参数范围内，那么就可以把它转化为一个 succinct dictionary。代价是：新的 random bits 用量变成

$$r'(n) = r(n) + (\log p(n)^{-1}) \cdot (\log \log n)^3,$$

新的 failure probability 变成

$$p'(n) = p(n)/\log \log n,$$

而新的附加空间冗余为

$$s'(n) = \frac{n \log n}{\log \log n}.$$

Applying Theorem 7 to Theorems 2 and 5, we get the following succinct versions of the theorems.

将 Theorem 7 分别应用到 Theorems 2 和 5, 就得到下面两个 succinct 版本的结论。

Corollary 8. *Let $0 < \epsilon < 1$ be a positive constant. There exists a $(r(n), p(n), s(n))$ -dictionary that uses $r(n) = O(n)$ random bits, that incurs a failure probability $p(n) = \exp(-n^{1-\epsilon})$, and that incurs an additive space overhead of $s(n) = O\left(\frac{n \log n}{\log \log n}\right)$ bits compared to the information-theoretical optimum.*

也就是说, 存在一种 succinct dictionary: 它使用 $O(n)$ 个 random bits, failure probability 为 $\exp(-n^{1-\epsilon})$, 并且相对于 information-theoretic optimum 只额外多出

$$O\left(\frac{n \log n}{\log \log n}\right)$$

比特空间。

Corollary 9. *There exists a $(r(n), p(n), s(n))$ -dictionary that uses $r(n) = O(\log n (\log \log n)^3) = \tilde{O}(\log n)$ random bits, that incurs a failure probability $p(n) = 1/\text{poly}(n)$, and that incurs an additive space overhead of $s(n) = O\left(\frac{n \log n}{\log \log n}\right)$ bits compared to the information-theoretical optimum.*

同样地, 也存在一种 succinct dictionary: 它只使用 $O(\log n (\log \log n)^3) = \tilde{O}(\log n)$ 个 random bits, failure probability 为 $1/\text{poly}(n)$, 并且相对于 information-theoretic optimum 的附加空间仍然只有

$$O\left(\frac{n \log n}{\log \log n}\right)$$

比特。

The rest of the section will be spent proving Theorem 7. We begin in Subsection 6.1 by presenting a reduction due to Raman and Rao [31], which transforms the problem of constructing a succinct dictionary into a different problem, which we call the **many-sets problem**. Then, in Subsection 6.2, we show how to solve the many-sets problem using an $(r(n), p(n), O(n \log n))$ -dictionary, while approximately preserving the randomness guarantees of the dictionary—an interesting feature of our solution is that it makes extensive use of the budget rotated trie constructed in Section 5.

本节余下内容都将用于证明 Theorem 7。首先, 在 Subsection 6.1 中, 我们会介绍 Raman 和 Rao [31] 的一个 reduction, 它把“构造 succinct dictionary”的问题转化为另一个问题, 我们将其称为 **many-sets problem**。然后, 在 Subsection 6.2 中, 我们再说明如何利用一个 $(r(n), p(n), O(n \log n))$ -dictionary 去解决 many-sets problem, 同时尽量保留原字典的随机化保证。值得注意的是, 我们的解法会大量利用第 5 节构造出的 budget rotated trie。

6.1 Reduction to the Many-Sets Problem

An important tool in our proof of Theorem 7 will be a reduction due to Raman and Rao [31]. This reduction transforms the problem of constructing a succinct dynamic dictionary into a different problem that we call the many-sets problem.

证明 Theorem 7 的一个关键工具, 是 Raman 和 Rao [31] 的一个 reduction。这个 reduction 会把“构造 succinct dynamic dictionary”的问题转化成另一个问题, 我们称之为 many-sets problem。

Let $\delta > 0$ be a small positive constant of our choice. The **many-sets problem**, with parameter δ , is defined as follows. Let $S_1, S_2, \dots, S_m$ be (dynamically changing) sets of $O(\log n)$ -bit key/value pairs, satisfying $\sum_i |S_i| = n$, $m \leq n / \text{polylog}(n)$, and $|S_i| \leq n^\delta$ for all $i \in [m]$, where n and m are permitted to evolve dynamically over time. We will use $\gamma_i = O(\log n)$ to denote the size of each key/value pair in S_i (so different S_i s may have differently sized key/value pairs).

取一个我们自行选择的小正常数 $\delta > 0$ 。带参数 δ 的 **many-sets problem** 定义如下: 我们要维护若干个动态变化的集合 $S_1, S_2, \dots, S_m$, 每个集合中的元素都是 $O(\log n)$ 比特的 key/value 对; 并满足 $\sum_i |S_i| = n$ 、 $m \leq n / \text{polylog}(n)$, 以及对所有 $i \in [m]$ 都有 $|S_i| \leq n^\delta$ 。这里 n 和 m 都允许随时间动态变化。我们用 $\gamma_i = O(\log n)$ 来表示 S_i 中每个 key/value 对的长度, 因此不同的 S_i 允许有不同长度的 key/value 对。

Any solution to the **many-sets problem** must support the following operations in constant time:

- INSERT(i, x, y), which inserts key/value pair (x, y) into S_i ;
- DELETE(i, x), which deletes x (and its corresponding value) from S_i ;
- and QUERY(i, x), which returns the value associated with x in S_i , or declares that $x \notin S_i$.

A many-sets solution is said to be an $(r(n), p(n), s(n))$ -**solution** if it uses $r(n)$ random bits, has failure probability $p(n)$ per insertion, and uses total space

$$\sum_i |S_i| \cdot (\gamma_i + O(\log |S_i|)) + s(n)$$

bits.

一个针对 **many-sets problem** 的解必须在常数时间内支持下列操作:

- INSERT(i, x, y): 把 key/value 对 (x, y) 插入到 S_i 中;
- DELETE(i, x): 从 S_i 中删除 key x 及其对应 value;
- QUERY(i, x): 查询 x 在 S_i 中是否存在, 若存在则返回其对应 value, 否则报告 $x \notin S_i$ 。

如果一个 many-sets 解使用了 $r(n)$ 个 random bits, 每次 insertion 的 failure probability 为 $p(n)$, 并且总空间为

$$\sum_i |S_i| \cdot (\gamma_i + O(\log |S_i|)) + s(n)$$

比特，那么我们称它是一个 $(r(n), p(n), s(n))$ -**solution**。

The following reduction is given (implicitly) in Section 3 of [31]:

Section 3 of [31] 中隐含给出了下面这个 reduction:

Theorem 10. *The problem of constructing a $(r(n), p(n), s(n) + n \log n / \log \log n)$ -dictionary reduces deterministically to the problem of constructing an $(r(n), p(n), s(n))$ -solution to the many-sets problem, where the parameter δ is a positive constant of our choice.*

也就是说，只要我们能解决 many-sets problem，并把其附加冗余控制在 $s(n)$ ，就可以 deterministic 地构造出一个 succinct dictionary，其附加冗余只多出 $n \log n / \log \log n$ 这一项；而参数 δ 可以由我们自由地选成任意正常数。

Understanding the reduction. Although we defer the full proof of Theorem 10 to [31], we take a moment to briefly describe the high-level structure here. The key insight is to make use of tries in a clever way (that differs substantially from how they are used elsewhere in this paper). Different nodes of the trie have different fanouts: if a node has $r > \text{polylog } n$ keys in it and has depth $O(1)$ in the trie, then it has fanout $r / \text{polylog } n$. If a node has either $r \leq \text{polylog } n$ keys in it, or has sufficiently large constant depth in the trie, then the node is a **leaf**, and the elements of the node correspond to a set S_i in the many-sets problem.

虽然 Theorem 10 的完整证明要留给 [31]，这里我们先简要说明它的高层结构。关键思想是：以一种很巧妙的方式使用 tries，而这种用法与本文其他地方对 tries 的使用方式非常不同。trie 中不同节点的 fanout 会不同：如果某个节点中有 $r > \text{polylog } n$ 个 keys，并且它在 trie 中的深度是 $O(1)$ ，那么它的 fanout 就取为 $r / \text{polylog } n$ 。如果某个节点中的 key 数量已经满足 $r \leq \text{polylog } n$ ，或者它在 trie 中的深度已经足够大，那么这个节点就会成为一个 **leaf**，而该叶节点中的元素就对应 many-sets problem 中的某个集合 S_i 。

Unlike for the data structures in this paper, the internal nodes of this trie are implemented using straightforward arrays: if a node has fanout f , it is implemented using an array of size f . Fortunately, by choosing the fanout f to be $r / \text{polylog } n$, where r is the number of elements stored by the node, one ensures that the arrays used to implement internal nodes have cumulative size $n / \text{polylog } n$.

与本文其他数据结构不同，这个 trie 的 internal nodes 是用非常直接的数组实现的：如果某个节点 fanout 为 f ，那么它就对应一个大小为 f 的数组。幸运的是，由于这里把 fanout 选成了 $r / \text{polylog } n$ ，其中 r 是该节点中元素的数量，因此可以保证所有 internal nodes 的数组加起来总大小只有 $n / \text{polylog } n$ 。

A key insight is that the trie allows for us to shave bits off of keys: if a node x has fanout f , then we can remove the high-order $\log f$ bits from each of the keys stored in x 's children (these bits are now stored implicitly in the trie path). Raman and Rao [31] show that, if a leaf has size $|S_i|$, then the length γ_i of each of the key/value pairs in S_i will satisfy $\gamma_i \leq c \log n - \log n - q \log |S_i| + O(\log \log n)$ bits for a positive constant q of our choice (depending on the maximum depth of the trie). This is

why, for the many-sets problem, it is okay to use (amortized) space $\gamma_i + O(\log |S_i|)$ bits per key in S_i .

这里另一个关键 insight 是：trie 能帮助我们 from keys 中“剃掉”一些比特。如果某个节点 x 的 fanout 是 f ，那么对存储在其 children 中的每个 key，我们都可以去掉高位的 $\log f$ 个比特，因为这些信息已经被隐含编码在 trie path 中了。Raman 和 Rao [31] 证明：若某个 leaf 的大小为 $|S_i|$ ，则 S_i 中每个 key/value 对的长度 γ_i 会满足

$$\gamma_i \leq c \log n - \log n - q \log |S_i| + O(\log \log n)$$

其中 q 是一个由我们选择的正常数，取决于 trie 的最大深度。正因为如此，在 many-sets problem 中，每个 S_i 的元素只花费 $\gamma_i + O(\log |S_i|)$ 比特空间是足够的。

We remark that, in [31], Theorem 10 was proved with *amortized* constant-time operations. The amortization came from the fact that, whenever a node x 's size changes substantially in the trie, it must be rebuilt; however, by spreading the rebuild across $\Theta(|S_i|)$ operations (that each modify the node x), the rebuild can trivially be deamortized to take $O(1)$ time per operation (and without compromising space efficiency, since each element is in only one version of the node at a time).

还要说明一点：在 [31] 中，Theorem 10 原本只证明了 *amortized* constant-time operations。摊还的来源在于：每当 trie 中某个节点 x 的大小变化过大时，它就必须被重建。不过，这种重建其实很容易去摊还化：只需把重建代价平摊到那 $\Theta(|S_i|)$ 次修改该节点 x 的操作上，就能把单次操作时间恢复到 $O(1)$ ，同时又不会破坏空间效率，因为任意时刻每个元素只会位于某个节点的一个版本中。

Before continuing, it is worth taking a moment to understand where the bounds $m \leq n / \text{polylog}(n)$ and $|S_i| \leq n^\delta$ (which are assumed in the many-sets problem) come from. The bound $m \leq n / \text{polylog}(n)$ comes from the fact that each internal node of the trie has fanout $r / \text{polylog}(n)$, where r is its size; since the trie has depth $O(1)$, this means that the total number of leaves (and thus the number of S_i s) is at most $O(n / \text{polylog}(n))$. The bound $|S_i| \leq n^\delta$ comes from the fact that, if a given S_i were to have size $> n^\delta$, then at each node on its root-to-leaf path, each element in the set would have $\delta \log n - O(\log \log n)$ bits shaved off; but assuming that the trie has sufficiently large constant depth, this means that *all* of the bits are shaved off from the elements of S_i , which is a contradiction.

在继续之前，还值得解释一下 many-sets problem 中两个关键约束的来源：为什么有 $m \leq n / \text{polylog}(n)$ ，以及为什么有 $|S_i| \leq n^\delta$ 。前者来自于 trie 的每个 internal node 都具有 $r / \text{polylog}(n)$ 的 fanout，其中 r 是该节点大小；再结合 trie 深度为 $O(1)$ ，就可推出 leaves 的总数，也就是 S_i 的总数，至多为 $O(n / \text{polylog}(n))$ 。后者则来自于这样一个事实：如果某个 S_i 的大小超过了 n^δ ，那么在它从 root 到 leaf 的路径上，每经过一个节点，该集合中的每个元素都会被剃掉 $\delta \log n - O(\log \log n)$ 个比特；只要 trie 的常数深度取得足够大，这就意味着 S_i 的元素最终会被剃掉所有比特，从而导致矛盾。

A starting place: Raman and Rao's solution to the many-sets problem. Raman and Rao [31] give a simple solution to the many-sets problem that incurs constant expected time per operation, and which will serve also as the starting point for our construction. At a high level, each

S_i is stored in a two-part structure, consisting of a *skeleton* A_i and a *storage array* B_i .⁹

Raman 和 Rao [31] 为 many-sets problem 给出了一个简单解法，其每次操作只保证 constant expected time，这个方案也将成为我们构造的起点。高层来看，每个 S_i 都用一个两部分结构来存储，分别是 *skeleton* A_i 和 *storage array* B_i 。¹⁰

The storage array B_i is logically implemented as a dynamically-resized array that stores the elements of S_i (both the keys and values) contiguously. The dynamic resizing of the $\{B_i\}$ s can be implemented (and deamortized) to take $O(1)$ time per operation while ensuring that, in aggregate, the B_i arrays use space within a factor of $1 + 1/\text{polylog}(n)$ of optimal [31] (see, also, [6], for a more detailed treatment of succinct dynamic arrays).

其中，storage array B_i 在逻辑上是一个动态扩缩容的数组，用来连续存储 S_i 的所有元素，包括 keys 和 values。对 $\{B_i\}$ 的动态扩缩容，可以通过去摊还化实现为每次操作 $O(1)$ 时间；同时还能保证所有 B_i 数组加起来所用空间至多只比最优解多一个 $1 + 1/\text{polylog}(n)$ 的因子 [31]。关于 succinct dynamic arrays 的更详细讨论，也可参考 [6]。

The skeleton A_i allows for queries to S_i to find the appropriate key/value in B_i . In particular, the skeleton is a dictionary that maps the $\Theta(\log |S_i|)$ -bit hash $h(x)$ for each key $x \in S_i$ to the index $j \in B_i$ at which x appears.¹¹

skeleton A_i 的作用，是帮助 query 操作在 B_i 中快速找到对应的 key/value 对。具体而言， A_i 本身是一个 dictionary：对每个 $x \in S_i$ ，它把一个长度为 $\Theta(\log |S_i|)$ 比特的 hash 值 $h(x)$ 映射到 x 在 B_i 中所处的位置 j 。¹²

The good news is that, within the skeleton A_i , both the keys (i.e., hashes $h(x)$) and the values (i.e., indices in B_i) are only $O(\log |S_i|)$ bits—this means that A_i can be implemented using any standard linear-space dictionary, without worrying about succinctness. The bad news is that there are two ways in which an insertion into A_i could fail: (1) the hash $h(x)$ of the element being inserted satisfies $h(x) = h(x')$ for some other $x' \in S_i$; or (2) the dictionary used to implement A_i fails.

好消息是，在 skeleton A_i 内部，无论是 keys (也就是 hashes $h(x)$) 还是 values (也就是 B_i 中的位置索引)，都只有 $O(\log |S_i|)$ 比特；这意味着我们完全可以用任意标准的 linear-space dictionary 去实现 A_i ，而不用担心 succinctness。坏消息则是：向 A_i 插入一个元素有两种失败方式。第一，待插入元素的 hash $h(x)$ 与已有某个 $x' \in S_i$ 满足 $h(x) = h(x')$ ，也就是发生碰撞；第二，实现 A_i 所用的底层 dictionary 本身发生 failure。

Assuming that A_i is implemented to have a $1/\text{poly}(|S_i|)$ failure probability (i.e., to be a w.h.p. dictionary), the probability of any given insertion into S_i failing is $1/\text{poly}(|S_i|)$. Of course, each $|S_i|$ can be arbitrarily small, which is why the data structure given in [31] offered constant expected-time operations, rather than a high-probability guarantee.

⁹The names *skeleton* and *storage array* are conventions that we are establishing here for ease of discussion, and were not used in the original paper [31].

¹⁰“skeleton”和“storage array”是为了便于讨论而在这里引入的称呼，原论文 [31] 并没有采用这两个术语。

¹¹As noted by [31], there is a subtle issue that one must be careful about for deletions. Whenever a deletion occurs, a gap is created in some position j of B_i . To remove this gap, one must move the final element x in B_i to position j . This means that we must also update the index that is stored for element x in A_i to be j .

¹²正如 [31] 所指出的，deletion 在这里有个细节需要小心处理：一旦删除发生， B_i 的某个位置 j 会出现一个空洞。要消除这个空洞，就必须把 B_i 最后一个元素 x 移动到位置 j ，与此同时也要把 A_i 中记录元素 x 的那个索引同步更新为 j 。

如果我们把 A_i 实现成一个 failure probability 为 $1/\text{poly}(|S_i|)$ 的 dictionary (也就是 w.h.p. dictionary), 那么任意一次向 S_i 插入元素的失败概率就是 $1/\text{poly}(|S_i|)$ 。但问题在于, 每个 $|S_i|$ 都可能非常小, 因此这种局部保证无法自动转化为关于全局规模 n 的 high-probability 保证。这也正是 [31] 中的数据结构只给出 constant expected-time, 而不是高概率保证的原因。

6.2 Proof of Theorem 7

By the reduction in the previous subsection (Theorem 10), it suffices for us to prove the following proposition:

由上一小节的 reduction (Theorem 10) 可知, 我们只需证明下面这个 proposition 即可。

Proposition 11. *Let ϵ be a small positive constant. Suppose that $r(n)$ and $p(n)$ are nondecreasing functions satisfying $r(n) \leq O(n)$ and $\exp(-n^{1-\epsilon}) \leq p(n) \leq 1/\text{polylog}(n)$. Given an $(r(n), p(n), n \log n)$ -dictionary, one can construct a $(r'(n), p'(n), s'(n))$ -solution to the many-sets problem, with parameter $\delta = 2\epsilon$, and with*

$$r'(n) = r(n) + (\log p(n))^{-1} \cdot (\log \log n)^3,$$

$$p'(n) = p(n/\log \log n),$$

and

$$s'(n) = \frac{n \log n}{\log \log n}.$$

这个 proposition 的意义是: 只要我们有一个 $(r(n), p(n), n \log n)$ -dictionary, 就能构造出一个针对 many-sets problem 的解, 其参数退化仅为

$$r'(n) = r(n) + (\log p(n))^{-1} \cdot (\log \log n)^3,$$

$$p'(n) = p(n/\log \log n),$$

并且空间冗余只有

$$s'(n) = \frac{n \log n}{\log \log n}.$$

We will prove Proposition 11 by adapting the skeleton/storage-array approach outlined in the previous section. The basic idea will be to implement each A_i as a budget rotated trie, and then to share random bits between A_i s in just the right way so that (1) the total number of random bits used is small; but (2) with good probability, only an $O(1/\log \log n)$ fraction of the elements present at any given moment will have experienced a failure when they were inserted. We then store the elements that experience failures in a backyard data structure implemented using the $(r(n), p(n), O(n \log n))$ -dictionary that we are given.

我们将通过改造上一小节中的 skeleton/storage-array 框架来证明 Proposition 11。基本思路是: 把每个 skeleton A_i 实现成一个 budget rotated trie, 然后用一种恰到好处的方式在不同 A_i

之间共享 random bits, 使得一方面, 总 random bits 数量仍然很小; 另一方面, 又能以足够高的概率保证任意时刻只有 $O(1/\log \log n)$ 比例的元素在插入时经历了 failure。凡是经历 failure 的元素, 我们都把它们存放到一个 backyard data structure 中, 而这个 backyard 则由题设给定的 $(r(n), p(n), O(n \log n))$ -dictionary 来实现。

Preliminaries on how to discuss the A_i s. Since the size of a given skeleton A_i may fluctuate over time, the skeleton A_i will need to be rebuilt every time that its size changes by a constant factor. These rebuilds can be deamortized to take constant time per operation (and since the dictionary A_i is permitted to use $O(|S_i| \log |S_i|)$ bits, it is acceptable for the rebuilds to add a constant-factor space overhead to A_i).

由于某个 skeleton A_i 的大小会随时间波动, 因此每当它的规模发生常数倍变化时, A_i 都需要重建。这种重建可以去摊还化, 使其平均到每次操作仍只需常数时间; 而且由于 A_i 本来就允许使用 $O(|S_i| \log |S_i|)$ 比特, 因此重建过程中带来的常数倍空间额外开销也是可以接受的。

At any given moment, define the **skeletal size** a_i of S_i to be the size that S_i was the most recent time that A_i was (logically) rebuilt. The skeletal size satisfies $a_i = \Theta(|S_i|)$ at all times, but has the convenient property that it only changes when a rebuild occurs. The skeletal size a_i will also be used to determine how we implement A_i (sets with different skeletal sizes may be implemented differently from one another)—this means that, each time A_i is rebuilt (and a_i changes), the implementation of A_i may also change.

在任意时刻, 我们把 S_i 的 **skeletal size** a_i 定义为: A_i 最近一次在逻辑上被重建时, S_i 当时的大小。始终都有 $a_i = \Theta(|S_i|)$, 但它有一个很方便的性质: 只有在 rebuild 真正发生时, a_i 才会变化。我们还会用 a_i 来决定 A_i 的实现方式, 因此不同 skeletal sizes 的集合可以采用不同实现; 这也意味着每次 A_i 重建、从而 a_i 发生变化时, A_i 的具体实现也可能一起变化。

Defining a backyard structure. An important component of the data structure will be a **backyard dictionary** T , which is used to store a small number of keys/values space inefficiently. We implement T using the $(r(n), p(n), n \log n)$ -dictionary that we are given, and we assume without loss of generality that $|T| \geq n/\log \log n$ at all times (if $|T| < n/\log \log n$, then we can pad it with dummy elements to bring the size up). This means that, at any given moment, T uses at most $O(r(n))$ random bits, has failure probability at most $O(p(n/\log \log n))$, and uses space at most

$$O((|T| + n/\log \log n) \cdot \log n)$$

bits. In order for T to meet the constraints of Proposition 11 we will need to establish that, with probability $1 - O(p'(n))$, we have

$$|T| \leq O(n/\log \log n) \tag{4}$$

at any given moment.

该数据结构中的另一个关键部件, 是一个 **backyard dictionary** T , 它用来以较低空间效率存储少量 keys/values。我们用题设中给定的 $(r(n), p(n), n \log n)$ -dictionary 来实现 T , 并且不失一般性地假设任意时刻都有 $|T| \geq n/\log \log n$; 如果不足这个值, 那么就用 dummy elements 把它补齐。因此, 在任意时刻, T 使用的 random bits 最多为 $O(r(n))$, 其 failure probability 至多为 $O(p(n/\log \log n))$, 并且总空间不超过

$$O(|T| + n/\log \log n) \cdot \log n$$

比特。为了让 T 满足 Proposition 11 的约束，我们接下来需要证明：以至少 $1 - O(p'(n))$ 的概率，在任意时刻都有

$$|T| \leq O(n/\log \log n) \quad (5)$$

It is worth taking a moment to describe precisely what information T stores for each key/value pair (x, y) that it stores belonging to a given skeleton A_i : it stores the pair (i, x) as a key and it stores y as a value. Since different A_i s have different key/value sizes (all of which are $\Theta(\log n)$), we pad the sizes of the keys/values to all be a fixed length $\Theta(\log n)$ in the backyard.¹³

这里还值得具体说明一下：对于属于某个 skeleton A_i 的任意 key/value 对 (x, y) ，如果它被放进了 backyard，那么 T 会把 (i, x) 作为 key，把 y 作为 value。由于不同的 A_i 可能拥有不同长度的 keys/values（尽管它们全都属于 $\Theta(\log n)$ ），因此在 backyard 中我们会把这些 keys/values 全部补齐到统一的 $\Theta(\log n)$ 长度。¹⁴

Storing small A_i s in the backyard automatically. Let c be a sufficiently large positive constant. We handle the A_i s satisfying $a_i \leq \log^c n$ by simply placing *all* of their elements in the backyard T . The number of such A_i s is trivially at most m , which by the definition of the many-sets problem, is at most $n/\text{polylog } n \leq O(n/\log^{2c} n)$. The number of elements that these A_i s contribute to the backyard is therefore at most $O(\log^c n \cdot n/\log^{2c} n) = o(n/\log^c n)$. Throughout the rest of the section, we will focus exclusively on A_i s satisfying $a_i > \log^c n$.

取一个足够大的正常数 c 。对于所有满足 $a_i \leq \log^c n$ 的 skeleton A_i ，我们采取最直接的做法：把它们的所有元素全部放进 backyard T 。这样的 A_i 数量显然至多为 m ，而根据 many-sets problem 的定义， $m \leq n/\text{polylog } n \leq O(n/\log^{2c} n)$ 。因此，这些小 A_i 向 backyard 贡献的元素总数最多只有

$$O(\log^c n \cdot n/\log^{2c} n) = o(n/\log^c n).$$

接下来本节余下部分中，我们只需关注那些满足 $a_i > \log^c n$ 的 skeleton。

Partitioning the remaining A_i s into groups $G_{j,k}$. We say that each A_i is in *category* $j = \lfloor \log a_i \rfloor$. Within each category j , we partition the A_i s (satisfying $a_i > \log^c n$) into

$$t_j = \frac{(\log \log n)^2 \log p(n)^{-1}}{j} \quad (6)$$

¹³As an additional subtlety, whenever an element is stored in the backyard, it should also be stored in the appropriate storage array B_i , that way whenever A_i is rebuilt, it can determine which of its elements are currently in the backyard, it can remove those elements from the backyard, and it can include those elements in the rebuild of A_i .

¹⁴还有一个细节：凡是存入 backyard 的元素，也必须同时保留在相应的 storage array B_i 中。这样一来，当 A_i 重建时，它就能知道自己的哪些元素当前仍在 backyard 中，从而把这些元素从 backyard 中删除，并在重建后的 A_i 中重新纳入考虑。

groups, $G_{j,1}, G_{j,2}, \dots, G_{j,t_j}$ such that, for each group $k \in [t_j]$,

$$\sum_{A_i \in G_{j,k}} a_i \leq O(n/t_j).$$

Note that such a partition is feasible only if and only if we can guarantee that $a_i \leq O(n/t_j)$ for each i —fortunately, this follows from

$$\begin{aligned} a_i \leq n^\delta &\leq O\left(n \cdot \frac{(\log \log n)^2}{jn^{1-\epsilon}}\right) && (\text{since } \epsilon = \delta/2) \\ &\leq O\left(n \cdot \frac{(\log \log n)^2 \log p(n)^{-1}}{j}\right) && (\text{since } \exp(-n^{1-\epsilon}) \leq p(n)) \\ &= O(n/t_j). \end{aligned}$$

For any given category j , the only difference between how we implement each of the groups $G_{j,1}, G_{j,2}, \dots, G_{j,t_j}$ is that each group $G_{j,k}$ is implemented using a different sequence $R_{j,k}$ of $\Theta(j \log j)$ random bits.

我们把每个 A_i 按照

$$j = \lfloor \log a_i \rfloor$$

划入一个 **category**。在每个固定的 category j 中，再把那些满足 $a_i > \log^c n$ 的 A_i 划分成

$$t_j = \frac{(\log \log n)^2 \log p(n)^{-1}}{j} \tag{7}$$

个 groups，记为 $G_{j,1}, G_{j,2}, \dots, G_{j,t_j}$ ，并要求每个 group $k \in [t_j]$ 都满足

$$\sum_{A_i \in G_{j,k}} a_i \leq O(n/t_j).$$

这样的划分可行，当且仅当每个 i 都满足 $a_i \leq O(n/t_j)$ 。幸运的是，这一点由下面的不等式链直接推出：

$$\begin{aligned} a_i \leq n^\delta &\leq O\left(n \cdot \frac{(\log \log n)^2}{jn^{1-\epsilon}}\right) && (\text{since } \epsilon = \delta/2) \\ &\leq O\left(n \cdot \frac{(\log \log n)^2 \log p(n)^{-1}}{j}\right) && (\text{since } \exp(-n^{1-\epsilon}) \leq p(n)) \\ &= O(n/t_j). \end{aligned}$$

对于固定的 category j ，groups $G_{j,1}, \dots, G_{j,t_j}$ 的唯一差别在于：每个 group $G_{j,k}$ 都会使用一段不同的 random-bit sequence $R_{j,k}$ ，其长度为 $\Theta(j \log j)$ 。

Implementing each A_i . We now describe how to implement a given skeleton $A_i \in G_{j,k}$ using the $\Theta(j \log j) = \Theta(\log a_j \log \log a_j) = \Theta(\log |A_j| \log \log |A_j|)$ random bits $R_{j,k}$. Recall that we store the A_i s satisfying $a_i \leq \log^c n$ in the backyard, so we need only focus here on A_i s satisfying $a_i > \log^c n$.

现在我们来证明：如何使用 $R_{j,k}$ 中的 $\Theta(j \log j) = \Theta(\log a_j \log \log a_j) = \Theta(\log |A_j| \log \log |A_j|)$ 个 random bits，去实现一个给定的 skeleton $A_i \in G_{j,k}$ 。回忆一下，所有满足 $a_i \leq \log^c n$ 的 A_i 已经被直接存到 backyard 中去了，因此这里我们只需关注 $a_i > \log^c n$ 的情形。

Define a hash function $h_{j,k}$ mapping the elements $x \in A_i$ to $\Theta(\log |A_i|)$ -bit string $h(x)$. The hash function is implemented using the family of hash functions given in Lemma 13 of Appendix A, meaning that the hash function uses $\Theta(\log |A_j|)$ random bits, and avoids collisions on A_i with probability $1 - 1/\text{poly}(|A_i|)$ (note that, since $a_i > \log^c n$, the precondition for the lemma is met). The skeleton A_i will map hashes $h_{j,k}(x)$, for $x \in A_i$, to indices in B_i .

首先定义一个 hash function $h_{j,k}$ ，把 A_i 中的元素 x 映射成一个 $\Theta(\log |A_i|)$ 比特的短串 $h(x)$ 。这个 hash function 来自附录 A 中 Lemma 13 给出的 family，因此它只消耗 $\Theta(\log |A_j|)$ 个 random bits，并且以 $1 - 1/\text{poly}(|A_i|)$ 的概率在 A_i 上避免碰撞（由于这里 $a_i > \log^c n$ ，该引理的前提已经满足）。于是 skeleton A_i 要做的事情，就是把每个 $x \in A_i$ 的 short hash $h_{j,k}(x)$ 映射到其在 B_i 中的索引。

We implement the dictionary A_i using a budget rotated trie (Theorem 5)—this uses $\Theta(\log |A_j| \log \log |A_j|)$ random bits and has failure probability $1/\text{poly}(|A_j|)$ per insertion. We remark that, in addition to making use of the probabilistic guarantees offered by the budget rotated trie, we will also be making use of the especially simple way in which the data structure experiences failures: the only possible failure mode is that one of the bins in the trie overflows. This will allow for us to gracefully handle when the A_j s fail: we store the element that experienced failure in a backup data structure, and we allow the A_j that caused the failure to continue as though that insertion never happened.

接着，我们用一个 budget rotated trie (Theorem 5) 来实现 dictionary A_i 。它需要 $\Theta(\log |A_j| \log \log |A_j|)$ 个 random bits，并且每次 insertion 的 failure probability 为 $1/\text{poly}(|A_j|)$ 。这里我们利用的不仅是 budget rotated trie 的概率保证，还有它 failure 模式的一个特别简单的性质：它唯一可能的 failure 方式，就是 trie 中某个 bin overflow。正因为 failure 如此单纯，我们才能在局部 skeleton 失效时优雅地处理：把出问题的那个元素转移到一个 backup data structure 中，然后让发生 failure 的 A_j 看起来就像“这次插入从未发生过”一样继续运行。

In more detail, there are two reasons that an insertion into A_i might fail: (1) the hash $h_{j,k}(x)$ of the element being inserted satisfies $h_{j,k}(x) = h_{j,k}(x')$ for some other $x' \in S_i$; or (2) the budget rotated trie used to implement A_i fails (i.e., the insertion would cause one of the bins in the trie to overflow). The probability of either of these events occurring at any given insertion is $1/\text{poly}(|A_i|) = 1/\text{poly}(2^j)$. Whenever an insertion into A_i fails, we store the key/value pair in the backyard data structure T instead. This means that our full data structure has two possible failure modes: the case where T itself fails, and the case where T becomes too large, violating (4).

更具体地说，向 A_i 做一次 insertion 可能失败的原因有两种。第一，待插入元素的 short hash 满足 $h_{j,k}(x) = h_{j,k}(x')$ ，也就是和某个已有元素 $x' \in S_i$ 发生了碰撞；第二，用来实现 A_i 的 budget rotated trie 本身发生了 failure，也就是这次插入会导致某个 bin overflow。这两种事件在任意一次 insertion 上发生的概率都只有 $1/\text{poly}(|A_i|) = 1/\text{poly}(2^j)$ 。每当一次向 A_i 的 insertion 失败时，我们就把对应的 key/value 对改存进 backyard T 。因此，对整体结构而言，只剩下两种真正的 failure 模式：要么 T 本身 failure，要么 T 长得太大，从而违反式 (4)。

Analyzing the probability of a failure. The backyard data structure T has failure probability at most $O(p(n/\log \log n))$, by design. Thus, our task is to bound the probability that (4) fails.

按照设计, backyard data structure T 本身的 failure probability 至多为 $O(p(n/\log \log n))$ 。因此, 剩下的核心任务就是控制式 (4) 被破坏的概率。

Lemma 12. *With probability $1 - \text{poly}(p(n))$, the category j contributes at most $O(n/(\log \log n)^2)$ elements to T , at any given moment.*

这个引理的意思是: 对任意固定的 category j , 以 $1 - \text{poly}(p(n))$ 的概率, 它在任意时刻向 backyard T 贡献的元素数都至多为 $O(n/(\log \log n)^2)$ 。这是后面全局控制 $|T|$ 的关键。

Proof. We have already established that the A_i s satisfying $a_i \leq \log^c n$ deterministically contribute at most $O(n/(\log \log n)^2)$ elements to T (because they deterministically *have* at most $O(n/(\log \log n)^2)$ elements). Thus we focus here on the A_i s satisfying $a_i > \log^c n$ (note that these are all in categories j satisfying $j > \Omega(\log \log n)$).

Now consider a category j satisfying $j = \Omega(\log \log n)$. As shorthand, we will use t to denote t_j and G_k to denote $G_{j,k}$. For each group G_k , $k \in [t]$, define X_k to be the number of elements that skeletons in G_k contribute to the backyard. We have that $X_k \leq O(n/t)$ deterministically; and, since each element x in each $A_i \in G_k$ has a $1/\text{poly}(2^j)$ probability of failure, we have that

$$\mathbb{E}[X_k] = \frac{n}{t \text{poly}(2^j)}. \quad (8)$$

Finally, since each of the groups $G_1, G_2, \dots, G_t$ use different random-bit sequences, the X_k s are independent. Defining $X'_k = X_k/\Theta(n/t)$, the number of elements that G_k contributes to the backyard can be expressed as $\Theta(\frac{n}{t} \cdot Y)$, where

$$Y = \sum_{k=1}^t X'_k.$$

The X'_k 's are independent random variables in the range $[0, 1]$ and, by (8),

$$\mathbb{E}[Y] = \sum_{k=1}^t \mathbb{E}[X'_k] = \sum_{k=1}^t \Theta(t/n) \mathbb{E}[X_k] = t/\text{poly}(2^j). \quad (9)$$

Applying a Chernoff bound to Y , we get

$$\Pr[Y > (1 + D)\mathbb{E}[Y]] \leq \left(\frac{e^D}{(1 + D)^{1+D}} \right)^{\mathbb{E}[Y]},$$

which for $D \geq \Omega(1)$ implies

$$\Pr[Y_j > D\mathbb{E}[Y]] \leq D^{-\Omega(D \cdot \mathbb{E}[Y])}.$$

Set

$$\begin{aligned} D &= \frac{t}{(\log \log n)^2 \mathbb{E}[Y]} \\ &= \frac{\text{poly}(2^j)}{(\log \log n)^2} && \text{(by (9))} \\ &\geq 2^{\alpha j} && \text{(since } j \geq \Omega(\log \log n)\text{),} \end{aligned}$$

where α is a positive constant of our choice. Then we have

$$\begin{aligned}
\Pr \left[Y_j > \frac{t}{(\log \log n)^2} \right] &\leq D^{-\Omega(D \cdot \mathbb{E}[Y])} \\
&= \exp(-\alpha j \cdot \Omega(D \cdot \mathbb{E}[Y])) && (\text{since } D \geq 2^{\alpha j}) \\
&= \exp(-\alpha j \cdot \Omega(t/(\log \log n)^2)) \\
&= \exp(-\alpha j \cdot \Omega(\log p(n)^{-1}/j)) && (\text{by (6)}) \\
&= \exp(-\alpha \cdot \Omega(\log p(n)^{-1})) \\
&= \text{poly}(p(n)).
\end{aligned}$$

Thus, we have with probability $1 - \text{poly}(p(n))$ that

$$Y \leq \frac{t}{(\log \log n)^2}.$$

The number of elements that category j contributes to the backyard is therefore at most

$$O\left(\frac{n}{t} \cdot Y\right) \leq O(n/(\log \log n)^2),$$

as desired. □

这个证明的主线并不复杂。首先，对那些满足 $a_i \leq \log^c n$ 的小 skeleton，我们已经知道它们 deterministic 地只会向 T 贡献 $O(n/(\log \log n)^2)$ 个元素，因此这里只需分析 $a_i > \log^c n$ 的部分。接着固定一个 category j ，把它内部的 groups 记成 G_k ，并把第 k 个 group 向 backyard 贡献的元素数记为 X_k 。由于每个 group 都使用彼此独立的 random-bit sequence，所以这些 X_k 相互独立；同时，每个 X_k 的期望都很小，而且最大值 deterministic 地受控。然后我们把 X_k 归一化成落在 $[0, 1]$ 区间内的随机变量，并对它们的和 Y 使用 Chernoff bound。只要把参数 D 精确设成一个与 t_j 和 $\mathbb{E}[Y]$ 相配的值，就能得到： Y 超过 $t/(\log \log n)^2$ 的概率只有 $\text{poly}(p(n))$ 。最终，这说明整个 category j 向 backyard 的贡献至多为

$$O\left(\frac{n}{t} \cdot \frac{t}{(\log \log n)^2}\right) = O\left(\frac{n}{(\log \log n)^2}\right),$$

也就是结论所需的界。

Putting the pieces together, the total size of T is $O(n/\log \log n)$ with probability

$$1 - O(p(n/\log \log n)) - (\log \log n) \cdot \text{poly}(p(n)) = 1 - O(p(n/\log \log n)).$$

把所有 category 的结论叠加起来，就得到：以

$$1 - O(p(n/\log \log n)) - (\log \log n) \cdot \text{poly}(p(n)) = 1 - O(p(n/\log \log n))$$

的概率，backyard T 的总大小都被控制在 $O(n/\log \log n)$ 。

Bounding the number of random bits. Finally, we count the number of random bits used by the data structure. The backyard uses $O(r(n))$ random bits, so it suffices to bound the number of random bits used by the skeletons in each category. Note that different categories can use the same random bits as one another (since we do not require independence between categories), so it suffices to bound the number of random bits used by any given category of skeletons. In category j , there are t_j groups, each of which uses $\Theta(j \log j) = O(j \log \log n)$ random bits. The total number of random bits used by the category is therefore

$$O(j(\log \log n)t_j) = O\left(j \log \log n \cdot \frac{(\log \log n)^2 \log p(n)^{-1}}{j}\right) = O((\log \log n)^3 \log p(n)^{-1}).$$

In total, the number of random bits used by the data structure is

$$O(r(n)) + O((\log \log n)^3 \log p(n)^{-1}).$$

This completes the proof of Proposition 11, and thus also the proof of Theorem 7.

最后还要统计 random bits 的数量。backyard 本身使用 $O(r(n))$ 个 random bits，因此只需估算每个 category 内 skeletons 的 random bits 开销即可。注意，不同 categories 之间并不要求彼此独立，所以它们甚至可以复用同一批 random bits；因此，我们只要控制住任意一个固定 category 的 random bits 用量就足够了。对于 category j ，它包含 t_j 个 groups，而每个 group 使用 $\Theta(j \log j) = O(j \log \log n)$ 个 random bits，因此该 category 总共消耗

$$O(j(\log \log n)t_j) = O\left(j \log \log n \cdot \frac{(\log \log n)^2 \log p(n)^{-1}}{j}\right) = O((\log \log n)^3 \log p(n)^{-1})$$

个 random bits。于是整个数据结构的 random bits 总量就是

$$O(r(n)) + O((\log \log n)^3 \log p(n)^{-1}).$$

这就完成了 Proposition 11 的证明，也同时完成了 Theorem 7 的证明。

A Appendix: Universe Reduction Using $O(\log n)$ Random Bits

In this section, we extend the budget rotated trie to support keys from a universe U of super-polynomial size. Throughout the section, we set $U = [2^u]$ for some $u = n^{o(1)}$, and we assume that machine words are $\Theta(u)$ bits.

本附录把 budget rotated trie 推广到来自 super-polynomial 大小 universe U 的 keys。全文中取 $U = [2^u]$ ，其中 $u = n^{o(1)}$ ，并假设 machine word 长度为 $\Theta(u)$ 比特。

To support large keys, the natural approach is to first hash elements from U to a smaller universe U' of polynomial size, and then to store the $\Theta(\log n)$ -bit keys in a hash table along with pointers to the full keys/values. Past work on load-balancing hash functions [7] has used a pair-wise

independent hash function $h : [2^u] \rightarrow [\text{poly}(n)]$ to perform this reduction. This requires the use of $\Theta(u)$ random bits, which when u is large, is significantly larger than $\log n \log \log n$.

为了支持大 key, 最自然的办法是先把 U 中的元素 hash 到一个更小的多项式规模 universe U' 中, 再把得到的 $\Theta(\log n)$ 比特 key 连同指向完整 key/value 的指针一起存入 hash table。以往关于 load-balancing hash functions 的工作 [7] 通常用一个 pair-wise independent 的 hash function $h : [2^u] \rightarrow [\text{poly}(n)]$ 来完成这一步约化。但这样会消耗 $\Theta(u)$ 个 random bits; 当 u 较大时, 这个量会远大于 $\log n \log \log n$ 。

An appealing alternative to using pairwise-independent hash functions would be to instead use Pagh's construction [28] (which, in turn, is based on an earlier construction by Fredman, Komlós, and Semerédi [14]) of $(1 + o(1))$ -universal hash functions that require only $O(\log n + \log \log u)$ random bits. The only minor problem with this construction is that it is not fully explicit. The construction requires access to a random prime number $p \in [\text{poly}(n)]$, but the only known time-efficient high-probability approaches to constructing such a prime number require $\omega(\log n \log \log n)$ random bits (see discussion in [13]).

一个看上去很吸引人的替代方案, 是使用 Pagh 的构造 [28], 它基于 Fredman、Komlós 和 Semerédi [14] 的更早构造, 能够得到只需 $O(\log n + \log \log u)$ 个 random bits 的 $(1 + o(1))$ -universal hash functions。这个构造唯一的小问题在于: 它还不是 fully explicit。因为它需要访问一个随机素数 $p \in [\text{poly}(n)]$, 而已知那些能够在时间上高效、并以高概率构造出这样一个素数的方法, 仍然需要 $\omega(\log n \log \log n)$ 个 random bits (参见 [13] 的讨论)。

Fortunately, this issue is relatively straightforward to solve. For completeness, we now give a construction for a simple family of hash functions that can be initialized in time $o(n)$ and used for universe reduction.

幸运的是, 这个问题本身并不难解决。为了叙述完整, 我们下面直接给出一个简单的 hash function family, 它能够在 $o(n)$ 时间内初始化, 并可用于 universe reduction。

Lemma 13. *Let $n > u^c$ for a sufficiently large positive constant c and let $S \subseteq [2^u]$ be a set of size n . Let $\mathcal{P}$ be the set of prime numbers in the range $[n^{2/c}]$. Select $p_1, p_2, \dots, p_{c^2}$ independently and uniformly at random from $\mathcal{P}$, and define the function $h : [2^u] \rightarrow [n^{2c}]$ by*

$$h(x) = (x \bmod p_1 p_2 \cdots p_{c^2}).$$

With probability $1 - 1/\text{poly}(n)$, h is injective on S .

这个引理说明: 只要随机选取若干个适当范围内的素数, 并把它们的乘积作为模数, 就能以 $1 - 1/\text{poly}(n)$ 的概率保证 h 在固定集合 S 上是单射。

Proof. The probability that $|h(S)| \neq S$ satisfies

$$\Pr[|h(S)| \neq S] \leq \sum_{s_1, s_2 \in S} \Pr[|s_1 - s_2| \text{ divisible by all of } p_1, p_2, \dots, p_{c^2}],$$

where s_1 and s_2 are implicitly taken to be distinct. Since the p_i s are independent, this is

$$\sum_{s_1, s_2 \in S} (\Pr[|s_1 - s_2| \text{ divisible by } p_1])^{c^2}.$$

The quantity $|s_1 - s_2|$ is an element of $U = [2^u]$, and can thus have at most u distinct prime factors. Therefore,

$$\begin{aligned} \Pr[|h(S)| \neq S] &\leq \sum_{s_1, s_2 \in S} \left(\frac{u}{|\mathcal{P}|} \right)^{c^2} \\ &\leq \sum_{s_1, s_2 \in S} \left(\frac{n^{1/c}}{|\mathcal{P}|} \right)^{c^2}. \end{aligned}$$

By the Prime Number Theorem, the set $\mathcal{P}$ of primes in the range $[n^{2/c}]$ has size $\Omega(n^{2/c}/\log n)$. Therefore,

$$\begin{aligned} \Pr[|h(S)| \neq S] &\leq \sum_{s_1, s_2 \in S} O\left(\frac{n^{1/c}}{n^{2/c}/\log n}\right)^{c^2} \\ &\leq O\left(\sum_{s_1, s_2 \in S} \left(\frac{\log n}{n^{1/c}}\right)^{c^2}\right) \\ &\leq O\left(\sum_{s_1, s_2 \in S} \frac{\log^{c^2} n}{n^c}\right) \\ &\leq O\left(\frac{n^2 \log^{c^2} n}{n^c}\right) \\ &\leq 1/\text{poly}(n). \end{aligned}$$

□

证明的核心很直接：若 h 在 S 上不是单射，那么一定存在两个不同元素 $s_1, s_2 \in S$ 使得它们在模 $p_1 p_2 \cdots p_{c^2}$ 意义下相等，也就是说差值 $|s_1 - s_2|$ 能被所有这些素数同时整除。由于 $|s_1 - s_2| < 2^u$ ，它最多只会有 u 个不同的素因子，因此从候选素数集合 $\mathcal{P}$ 中反复独立地选出 c^2 个素数后，它们恰好全都整除某个固定差值的概率非常小。最后再对所有元素对做一次 union bound，并利用 Prime Number Theorem 给出 $|\mathcal{P}|$ 的下界，就能把总失败概率压到 $1/\text{poly}(n)$ 。

Since all of the prime numbers in $[n^\epsilon]$ can be enumerated in time $O(n^{2\epsilon})$, we get the following corollary:

由于区间 $[n^\epsilon]$ 中的所有素数都可以在 $O(n^{2\epsilon})$ 时间内枚举出来，因此我们立刻得到下面的 corollary。

Corollary 14. *Let $u = n^{o(1)}$. For any constant $\delta > 0$, there exists an explicit family $\mathcal{H}$ of constant-time hash functions $h : [2^u] \rightarrow [\text{poly}(n)]$ such that (a) a random function $h \in \mathcal{H}$ can be constructed in time $O(n^\delta)$ using $O(\log n)$ random bits; and (b) for any fixed set $S \subseteq U$ of size n , and for a random $h \in \mathcal{H}$, we have that $|h(U)| = |U|$ with probability $1 - 1/\text{poly}(n)$.*

也就是说，当 $u = n^{o(1)}$ 时，对任意常数 $\delta > 0$ ，都存在一个 explicit 的 constant-time hash function family $\mathcal{H}$ ：从中随机抽取一个 h 只需要 $O(\log n)$ 个 random bits，并可在 $O(n^\delta)$ 时间内构

造出来；同时，对任意固定大小为 n 的集合 $S \subseteq U$ ，随机 h 以至少 $1 - 1/\text{poly}(n)$ 的概率在 S 上保持单射。

We can use Corollary 14 to construct a version of the budget rotated trie that supports large universes.

利用 Corollary 14，我们就能构造出一个支持大 universe 的 budget rotated trie。

Theorem 15. *Let $u = n^{o(1)}$, suppose that keys/values are u bits, and assume a machine word of size at least $\Omega(u)$ bits. The budget rotated trie uses $O(\log n \log \log n)$ random bits, it uses $O(nu)$ bits of space, and it supports insert/delete/query operations on up to n keys/values at a time. The data structure can be initialized in time $O(n^\epsilon)$, for a positive constant ϵ of our choice, and each insert/delete/query operation takes constant time with probability $1 - 1/\text{poly}(n)$.*

这个定理说明：当 $u = n^{o(1)}$ ，并且 keys/values 长度为 u 比特、machine word 长度至少为 $\Omega(u)$ 比特时，budget rotated trie 仍然只使用 $O(\log n \log \log n)$ 个 random bits，占用 $O(nu)$ 比特空间，并支持同时容纳最多 n 个 key/value 对。其初始化时间可做到 $O(n^\epsilon)$ （其中 ϵ 是我们任选的正常数），而每次 insert/delete/query 操作都以 $1 - 1/\text{poly}(n)$ 的概率在常数时间内完成。

To eliminate the $O(n^\epsilon)$ initialization cost, we can also construct a dynamic version of the same data structure, where there is some upper bound N on the data structure's size, but where the true size n changes over time. Every time that the data structure's size changes by a constant factor, we rebuild it based on the new value of n . Each rebuild takes time $O(n)$ (with high probability in n), but the cost of a rebuild can be spread across $\Theta(n)$ operations. The properties of this new data structure can be summarized with the following corollary.

如果还想进一步去掉那 $O(n^\epsilon)$ 的初始化代价，我们也可以构造这个数据结构的动态版本：假设有一个总上界 N ，而真实大小 n 会随时间变化。每当数据结构大小发生常数倍变化时，我们就根据新的 n 重新 build 一次。单次 rebuild 以 high probability 需要 $O(n)$ 时间，但这个代价可以均摊到接下来的 $\Theta(n)$ 次操作上。于是，这个动态版本的性质就可以概括为下面的 corollary。

Corollary 16. *Let $u = N^{o(1)}$, suppose that keys/values are u bits, and assume a machine word of size at least $\Omega(u)$ bits. The dynamic budget rotated trie uses $O(\log N \log \log N)$ random bits and supports insert/delete/query operations on up to N keys/values at a time. If it is storing n key/value pairs, then it uses $O(nu)$ bits of space, and each insert/delete/query operation takes constant time with probability $1 - 1/\text{poly}(n)$.*

也就是说，若 $u = N^{o(1)}$ ，并且 keys/values 长度为 u 比特、machine word 至少有 $\Omega(u)$ 比特，那么 dynamic budget rotated trie 只使用 $O(\log N \log \log N)$ 个 random bits，就能支持至多 N 个 key/value 对的 insert/delete/query 操作；在任意时刻若它当前存储了 n 个 key/value 对，那么其空间开销为 $O(nu)$ 比特，并且每次操作都以 $1 - 1/\text{poly}(n)$ 的概率在常数时间内完成。

Acknowledgements.

The author is grateful to Martin Dietzfelbinger for helpful pointers to prior work, and would also like to thank Michael A. Bender and Martín Farach-Colton for a number of insightful technical discussions.

作者感谢 Martin Dietzfelbinger 在相关前人工作方面提供的有益指引，也感谢 Michael A.

Bender 和 Martín Farach-Colton 进行的多次富有洞见的技术讨论。

References

- [1] Alfred V Aho and John E Hopcroft. *The design and analysis of computer algorithms*. Pearson Education India, 1974.
- [2] Mihir Bellare and John Rompel. Randomness-efficient oblivious sampling. In *Proceedings 35th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 276–287. IEEE, 1994.
- [3] Michael A Bender, Alex Conway, Martín Farach-Colton, William Kuszmaul, and Guido Tagliavini. All-purpose hashing. *arXiv preprint arXiv:2109.04548*, 2021.
- [4] Michael A. Bender, Martín Farach-Colton, John Kuszmaul, William Kuszmaul, and Mingmou Liu. On the optimal time/space tradeoff for hash tables. In *Proceedings of the Fifty-Fourth Annual ACM Symposium on Theory of Computing (STOC)*, 2022.
- [5] Jon Bentley. *Programming pearls*. Addison-Wesley Professional, 2016.
- [6] Andrej Brodnik, Svante Carlsson, Erik D Demaine, J Ian Ian Munro, and Robert Sedgewick. Resizable arrays in optimal time and space. In *Workshop on Algorithms and Data Structures*, pages 37–48. Springer, 1999.
- [7] L Elisa Celis, Omer Reingold, Gil Segev, and Udi Wieder. Balls and bins: Smaller hash families and faster evaluation. In *Proceedings of the 2011 IEEE 52nd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 599–608, 2011.
- [8] M Dietzfelbinger, A Karlin, K Mehlhorn, FM auf der Heide, H Rohnert, and RE Tarjan. Dynamic perfect hashing: upper and lower bounds. In *Proceedings of the 29th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 524–531, 1988.
- [9] Martin Dietzfelbinger and Friedhelm Meyer auf der Heide. A new universal class of hash functions and dynamic hashing in real time. In *International Colloquium on Automata, Languages, and Programming (ICALP)*, pages 6–19. Springer, 1990.
- [10] Martin Dietzfelbinger, Joseph Gil, Yossi Matias, and Nicholas Pippenger. Polynomial hash functions are reliable. In *International Colloquium on Automata, Languages, and Programming*, pages 235–246. Springer, 1992.
- [11] Martin Dietzfelbinger and Philipp Woelfel. Almost random graphs with simple hash functions. In *Proceedings of the Thirty-Fifth Annual ACM Symposium on Theory of Computing (STOC)*, pages 629–638, 2003.
- [12] Devdatt P Dubhashi and Alessandro Panconesi. *Concentration of measure for the analysis of randomized algorithms*. Cambridge University Press, 2009.
- [13] Pierre-Alain Fouque and Mehdi Tibouchi. Close to uniform prime number generation with fewer random bits. In *International Colloquium on Automata, Languages, and Programming (ICALP)*, pages 991–1002. Springer, 2014.

- [14] Michael L. Fredman, Janos Komlos, and Endre Szemerédi. Storing a sparse table with $o(1)$ worst case access time. In *Proceedings of the 23rd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 165–169. IEEE Computer Society, 1982.
- [15] Michael L Fredman and Dan E Willard. Blasting through the information theoretic barrier with fusion trees. In *Proceedings of the twenty-second annual ACM Symposium on Theory of Computing (STOC)*, pages 1–7, 1990.
- [16] Michael T Goodrich, Daniel S Hirschberg, Michael Mitzenmacher, and Justin Thaler. Fully de-amortized cuckoo hashing for cache-oblivious dictionaries and multimaps. *arXiv preprint arXiv:1107.4378*, 2011.
- [17] Michael T Goodrich, Daniel S Hirschberg, Michael Mitzenmacher, and Justin Thaler. Cache-oblivious dictionaries and multimaps with negligible failure probability. In *Mediterranean Conference on Algorithms*, pages 203–218. Springer, 2012.
- [18] Torben Hagerup, Peter Bro Miltersen, and Rasmus Pagh. Deterministic dictionaries. *Journal of Algorithms*, 41(1):69–85, 2001. URL: <https://www.sciencedirect.com/science/article/pii/S019667740191171X>, doi:<https://doi.org/10.1006/jagm.2001.1171>.
- [19] Eyal Kaplan, Moni Naor, and Omer Reingold. Derandomized constructions of k -wise (almost) independent permutations. *Algorithmica*, 55(1):113–133, 2009.
- [20] Mingmou Liu, Yitong Yin, and Huacheng Yu. Succinct filters for sets of unknown sizes. In *47th International Colloquium on Automata, Languages, and Programming (ICALP)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2020.
- [21] Michael Luby and Charles Rackoff. How to construct pseudorandom permutations from pseudorandom functions. *SIAM Journal on Computing*, 17(2):373–386, 1988.
- [22] Colin McDiarmid. On the method of bounded differences. *Surveys in combinatorics*, 141(1):148–188, 1989.
- [23] Raghu Meka, Omer Reingold, Guy N Rothblum, and Ron D Rothblum. Fast pseudorandomness for independence and load balancing. In *International Colloquium on Automata, Languages, and Programming (ICALP)*, pages 859–870. Springer, 2014.
- [24] Moni Naor and Omer Reingold. On the construction of pseudorandom permutations: Luby—Rackoff revisited. *Journal of Cryptology*, 12(1):29–66, 1999.
- [25] Anna Ostlin and Rasmus Pagh. Uniform hashing in constant time and linear space. In *Proceedings of the thirty-fifth annual ACM symposium on Theory of Computing (STOC)*, pages 622–628, 2003.
- [26] Anna Pagh and Rasmus Pagh. Uniform hashing in constant time and optimal space. *SIAM Journal on Computing*, 38(1):85–96, 2008.
- [27] Rasmus Pagh. Faster deterministic dictionaries. In *Proceedings of the Eleventh Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 487–493, USA, 2000.
- [28] Rasmus Pagh. Dispersing hash functions. *Random Structures & Algorithms*, 35(1):70–82, 2009.
- [29] Mihai Pătraşcu and Mikkel Thorup. The power of simple tabulation hashing. *Journal of the ACM (JACM)*, 59(3):1–50, 2012.

- [30] Mihai Patrascu and Mikkell Thorup. Dynamic integer sets with optimal rank, select, and predecessor search. In *2014 IEEE 55th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 166–175, 2014.
- [31] Rajeev Raman and Satti Srinivasa Rao. Succinct dynamic dictionaries and trees. In *Automata, Languages and Programming*, pages 357–368, Berlin, Heidelberg, 2003. Springer Berlin Heidelberg.
- [32] Omer Reingold, Ron D Rothblum, and Udi Wieder. Pseudorandom graphs in data structures. In *International Colloquium on Automata, Languages, and Programming (ICALP)*, pages 943–954. Springer, 2014.
- [33] Milan Ružić. Uniform deterministic dictionaries. *ACM Trans. Algorithms*, 4(1), March 2008. doi:10.1145/1328911.1328912.
- [34] A Siegel. On universal classes of fast high performance hash functions, their time-space tradeoff, and their applications. In *Proceedings of the 30th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 20–25, 1989.
- [35] Alan Siegel. On universal classes of extremely random constant-time hash functions. *SIAM Journal on Computing*, 33(3):505–543, 2004.
- [36] Rajamani Sundar. A lower bound for the dictionary problem under a hashing model. In *Proceedings 32nd Annual Symposium of Foundations of Computer Science (FOCS)*, pages 612–621, 1991. doi:10.1109/SFCS.1991.185427.
- [37] Mikkell Thorup. Mihai Pătraşcu: Obituary and open problems. *Bulletin of EATCS*, 1(109), 2013.